

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук  
Образовательная программа «Программная инженерия»

УДК 004.422.832

УТВЕРЖДАЮ  
Академический руководитель  
образовательной программы  
«Программная инженерия»,  
старший преподаватель департамента  
программной инженерии

\_\_\_\_\_  
Н.А. Павлочев  
«16» \_\_\_\_\_ 05 \_\_\_\_\_ 2025 г.

**Выпускная квалификационная работа**

на тему «Серверная часть умного корпуса текстов СМИ»

по направлению подготовки 09.03.04 «Программная инженерия»

Научный руководитель

Доцент департамента программной  
инженерии факультета компьютерных наук

Должность, место работы

\_\_\_\_\_  
канд. комп. наук.

ученая степень, ученое звание

\_\_\_\_\_  
А.А. Мицюк

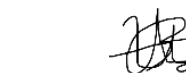
\_\_\_\_\_  
И.О. Фамилия

 \_\_\_\_\_  
16.05.2025  
Подпись, Дата

Выполнил

студент группы БПИ217  
4 курса бакалавриата  
образовательной программы  
«Программная инженерия»

\_\_\_\_\_  
А.В. Цейтин  
И.О. Фамилия

 \_\_\_\_\_  
16.05.2025  
Подпись, Дата

**Москва 2025**

## Реферат

На протяжении всей истории банковская сфера была тесно связана со скоростью реагирования на изменения в мире, а также адаптацию к ним, по этой причине необходимо искать любую возможность для увеличения скорости работы и продуктивности в этой области. Сегодня существует большое множество информационных источников, прочитав и проанализировав, которые можно четко осознать текущую обстановку в мире, однако для этого необходимо потратить множество времени на сбор новостных статей, их фильтрацию, вычленение наиболее важной информации, а также формирование общей картины на основе прочитанного. Все перечисленное уже по отдельности является довольно трудоемким и время затратным процессом, а вместе превращается в серьезную проблему, на которую приходится ежедневно выделять множество сил и человеческих ресурсов. Именно с такой проблемой и столкнулись сотрудники Т-банка, из-за чего появилась потребность автоматизировать данные процессы.

Умный корпус текстов СМИ помогает собирать все статьи из определенного набора источников, оформляет по каждой из них краткую выжимку, помечает подходящими тэгами из заранее определенного списка, формирует новостные дайджесты по выбранным темам, с возможностью их дальнейшей рассылки, а также предоставляет гибкий инструментарий для работы с имеющимися материалами. Данная же работа является реализацией сервисной части упомянутого веб-приложения, которая берет на себя большую часть бизнес-логики всего приложения, хранение и управление данными, а также настройка их взаимодействия с другими частями веб-приложения.

Ключевые слова: серверная часть, умный корпус текстов СМИ, новостные статьи, новостные дайджесты, точный и синтаксический поиск, аннотации.

Работа содержит 70 страницы, 3 главы, 72 иллюстраций, 12 источников.

## **Abstract**

Throughout history, banking has been closely linked to the speed of responding to and adapting to changes in the world, for this reason it is necessary to look for any opportunity to increase the speed of work and productivity in this area. Today there are a lot of information sources, reading and analyzing which you can clearly understand the current situation in the world, but for this you need to spend a lot of time to collect news articles, filtering them, selecting the most important information, as well as forming a general picture based on what you read. All of the above individually is a rather labor-intensive and time-consuming process, but together it turns into a serious problem, which requires a lot of effort and human resources to be allocated on a daily basis. T-Bank employees faced just such a problem, which led to the need to automate these processes.

The smart media text corpus helps to collect all articles from a certain set of sources, creates a brief extract for each of them, tags them with appropriate tags from a predetermined list, forms news digests on selected topics with the possibility of their further distribution, and also provides a flexible toolkit for working with existing materials. This work is the implementation of the service part of the mentioned web application, which takes care of most of the business logic of the whole application, data storage and management, as well as setting up their interaction with other parts of the web application.

Keywords: server side, smart corpus of media texts, news articles, news digests, accurate and syntactic search, annotations.

The work contains 70 pages, 3 chapters, 72 illustrations, and 12 source

## Содержание

|                                                                                              |    |
|----------------------------------------------------------------------------------------------|----|
| РЕФЕРАТ .....                                                                                | 2  |
| ABSTRACT .....                                                                               | 3  |
| СОДЕРЖАНИЕ.....                                                                              | 4  |
| ОСНОВНЫЕ ПОНЯТИЯ И ОПРЕДЕЛЕНИЯ .....                                                         | 6  |
| ВВЕДЕНИЕ .....                                                                               | 7  |
| 1 ПРЕДМЕТНАЯ ОБЛАСТЬ И СУЩЕСТВУЮЩИЕ РЕШЕНИЯ .....                                            | 8  |
| 1.1 Описание предметной области.....                                                         | 8  |
| 1.2 Обзор агрегаторов и средств автоматического мониторинга СМИ .....                        | 9  |
| 1.2.1 SmartNews .....                                                                        | 9  |
| 1.2.2 Feedly .....                                                                           | 9  |
| 1.2.3 Seldon News .....                                                                      | 9  |
| 1.2.4 uCrawler .....                                                                         | 10 |
| 1.2.5 Flipboard.....                                                                         | 10 |
| 1.2.6 Рамблер Новости .....                                                                  | 11 |
| 1.2.7 Яндекс Дзен.....                                                                       | 11 |
| Выводы по главе .....                                                                        | 13 |
| 2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ, ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ .....                                      | 14 |
| 2.1 Функциональные требования к программе.....                                               | 14 |
| 2.2 Описание архитектуры приложения.....                                                     | 14 |
| 2.3 Диаграмма прецедентов.....                                                               | 16 |
| 2.4 Архитектура решения .....                                                                | 18 |
| 2.5 Выбор технологий реализации.....                                                         | 18 |
| 2.6 Выбор базы данных .....                                                                  | 18 |
| 2.7 Архитектура базы данных в целом и особенности хранения дайджестов и подписок на них..... | 20 |
| Выводы по главе .....                                                                        | 22 |
| 3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ.....                                                        | 23 |
| 3.1 Функционал, связанный с управлением пользователями и авторизацией .....                  | 23 |
| 3.1.1 Регистрация пользователя .....                                                         | 24 |
| 3.1.2 Авторизация пользователя .....                                                         | 25 |
| 3.1.3 Проверка разрешений пользователя .....                                                 | 27 |
| 3.2 Функционал, связанный с обработкой, хранением и управлением новостными статьями.....     | 29 |
| 3.2.1 Создание нового документа .....                                                        | 31 |
| 3.2.2 Получение списка всех документов .....                                                 | 31 |
| 3.2.3 Получение документа по id .....                                                        | 33 |
| 3.2.4 Редактирование документа по id.....                                                    | 34 |

|        |                                                                            |    |
|--------|----------------------------------------------------------------------------|----|
| 3.2.5  | Удаление документа по id .....                                             | 38 |
| 3.2.6  | Оценка документа.....                                                      | 41 |
| 3.2.7  | Поиск документов .....                                                     | 43 |
| 3.2.8  | Добавление документа в папку .....                                         | 45 |
| 3.2.9  | Удаление документа из папки.....                                           | 47 |
| 3.2.10 | Добавление документа в избранное .....                                     | 48 |
| 3.2.11 | Удаление документа из избранного.....                                      | 50 |
| 3.2.12 | Создание аннотации в документе .....                                       | 51 |
| 3.2.13 | Удаление аннотации из документа .....                                      | 53 |
| 3.2.14 | Создание комментария к документу .....                                     | 55 |
| 3.2.15 | Удаление комментария к документу .....                                     | 56 |
| 3.3    | Функционал, связанный с управлением жизненным циклом тэгов .....           | 58 |
| 3.3.1  | Создание тэга .....                                                        | 58 |
| 3.3.2  | Удаление тэга по id.....                                                   | 59 |
| 3.4    | Функционал, связанный с управлением жизненным циклом папок .....           | 60 |
| 3.4.1  | Создание папки .....                                                       | 60 |
| 3.4.2  | Удаление папки по названию .....                                           | 62 |
| 3.5    | Функционал, связанный с управлением жизненным циклом подписок на дайджесты | 64 |
| 3.5.1  | Создание подписки.....                                                     | 64 |
|        | Выводы по главе .....                                                      | 68 |
|        | ЗАКЛЮЧЕНИЕ.....                                                            | 69 |
|        | ИСТОЧНИКИ .....                                                            | 70 |

## Основные понятия и определения

Микросервис (микросервисная архитектура) – это способ организации программного обеспечения, при котором система состоит из множества независимых сервисов, взаимодействующих через API. Каждый микросервис выполняет ограниченную бизнес-логику, разрабатывается, разворачивается и масштабируется отдельно, что повышает гибкость, отказоустойчивость и удобство сопровождения системы

Классификация (в контексте машинного обучения) – это задача, при которой модель предсказывает категорию или класс для входных данных

Dashboard (Дашборд) — это визуальный интерфейс, который отображает ключевые показатели и данные в реальном времени, часто в виде графиков, диаграмм и таблиц

LLM (Large Language Model, большой языковой модель) – это тип модели машинного обучения, основанный на нейросетях и обученный на больших объемах текстовых данных

REST API - Архитектурный стиль для создания веб-сервисов, использующий HTTP-запросы для взаимодействия между клиентом и сервером. Данные передаются в формате JSON или XML

JSON (JavaScript Object Notation) - Формат обмена данными, основанный на текстовом представлении структурированных данных. Широко используется в веб-разработке для передачи данных между клиентом и сервером

Java Spring Boot - Фреймворк для создания веб-приложений на языке Java. Упрощает разработку за счет встроенных механизмов автоконфигурации и готовых решений для работы с базами данных, безопасностью и другими компонентами [9][10]

PostgreSQL - Реляционная система управления базами данных (СУБД) с открытым исходным кодом. Поддерживает сложные запросы, транзакции и полнотекстовый поиск [8]

OpenSearch - Поисковая система с открытым исходным кодом, основанная на Elasticsearch. Используется для выполнения полнотекстового поиска и анализа больших объемов данных [11][12]

Full-text search (полнотекстовый поиск) - Технология поиска текстовой информации, позволяющая находить документы по ключевым словам или фразам. Используется в базах данных и поисковых системах

Backend – Серверная часть приложения, отвечающая за логику, обработку данных и взаимодействие с базой данных

## **Введение**

Данная выпускная квалификационная работа посвящена реализации серверной части системы под названием «Умный Корпус Текстов СМИ». Серверная часть является ключевым компонентом веб-приложения, поскольку она обеспечивает взаимодействие между всеми остальными модулями системы. От корректности и эффективности ее работы напрямую зависит успех всего проекта в целом. Основное внимание в работе уделено проектированию и реализации архитектуры серверной части, включая создание объектов базы данных, настройку их взаимодействия и обеспечение функциональности, необходимой для работы с новостными статьями.

Основной функционал приложения сконцентрирован вокруг новостных статей, с ними взаимодействуют практически все остальные сущности базы данных. Однако основное предназначение новостей – дальнейшее формирование на их основе новостных дайджестов, с помощью которых пользователям не придется самостоятельно перебирать все статьи, поскольку вся основная информация будет находиться в дайджесте.

Также важным аспектом работы является управление пользователями. Поскольку доступ к подпискам, аннотациям, комментариям, дайджестам, папкам с новостными статьями, а также к некоторому функционалу, четко разграничивается на основе авторизации пользователей, то ей было уделено особое внимание несмотря на то, что данный функционал можно считать вспомогательным.

На основе вышеизложенного можно выделить следующие задачи, которые решаются в рамках данной работы:

- 1) Анализ рынка и существующих решений,
- 2) Формулирование требований к системе,
- 3) Выбор технологий и инструментов реализации,
- 4) Проектирование архитектуры системы,
- 5) Обеспечение взаимодействия с другими модулями,
- 6) Реализация серверной части корпуса текстов СМИ,
- 7) Составление технической документации.

## **1 Предметная область и существующие решения**

### **1.1 Описание предметной области**

Успех в банковской сфере во многом зависит от способности быстро адаптироваться к новым условиям, что требует постоянного мониторинга актуальной информации. В прошлом, банкам приходилось постоянно конкурировать друг с другом за получение доступа к самой свежей и достоверной информации, что было вызвано ее дефицитом и низкой скоростью передачи.

Однако, сегодня ситуация кардинально поменялась. Дефицит сменился на профицит и приходится задумываться не о поиске и скорости поставки информации, а об ее обработке. Это обусловлено развитием информационных технологий, в особенности Интернета, с помощью которого можно в считанные секунды получить огромные объемы данных. Обработка таких массивов особенно важна в банковской сфере, где не позволительно тратить время на лишние размышления.

Для решения данной проблемы на рынке появились специализированные сервисы – агрегаторы новостей. Они предоставляют широкий функционал для сбора, фильтрации и поиска новостных статей, что значительно упрощает обработку информации. В некоторых из агрегаторов даже присутствует формирование новостных дайджестов, с возможностью персонализированной рассылки, что должно еще сильнее сократить время затраты пользователей.

Умный корпус текстов СМИ также попадает под категорию агрегатора новостей, однако является более специализированным под нужды сотрудников Т-банка, например позволяя настроить список источников и тэгов, по которым будут классифицироваться статьи. Однако это не весь доступный функционал, приложение также располагает и исследовательским инструментарием, позволяя формировать аннотации, оставлять комментарии, группировать новостные статьи по папкам, а также оценивать работу сервисов генерации новостных дайджестов и классификации статей, что возможно использовать в дальнейшем дообучении моделей, применяемых для данного функционала.

Таким образом можно прийти к выводу, что актуальность данного проекта обусловлена нуждами банков, в особенности сотрудников Т-банка, в гибком инструментарии, способном упростить работу с обработкой больших объемов данных.

## **1.2 Обзор агрегаторов и средств автоматического мониторинга СМИ**

### **1.2.1 SmartNews**

SmartNews представляет собой мобильное приложение для агрегации новостей, которое предлагает пользователям доступ к актуальной информации из более чем 200 локальных источников. Основная цель сервиса — предоставлять свежие новости из проверенных изданий, адаптируя контент под индивидуальные предпочтения пользователей. Пользователи могут настраивать ленту, выбирая интересующие темы и источники, а также сравнивать различные точки зрения на одно и то же событие [1].

Однако приложение имеет ограниченный охват, работая преимущественно на рынках Японии и США. Оно недоступно для скачивания в других странах, включая Россию, что существенно сужает его аудиторию. Кроме того, SmartNews функционирует исключительно как новостной агрегатор, не предлагая дополнительных функций, таких как аналитика или инструменты для работы с контентом.

### **1.2.2 Feedly**

Feedly — один из ведущих сервисов для агрегации новостного контента, запущенный в 2008 году. Он позволяет пользователям собирать статьи, новости и обновления из различных источников в единой ленте, что делает его удобным инструментом для отслеживания актуальных событий. Основной функционал включает подписку на RSS-каналы, организацию новостей в тематические коллекции и создание кратких выжимок из статей [2].

Важным преимуществом Feedly является наличие API, что позволяет разработчикам интегрировать сервис в собственные решения. Пользователи с платной подпиской получают доступ к расширенной аналитике, включая отслеживание популярных тем и ключевых слов. Однако бесплатная версия имеет ограниченный функционал, а отсутствие дашбордов снижает возможности использования сервиса для исследовательских целей.

### **1.2.3 Seldon News**

Seldon News — это российский агрегатор новостей, разработанный компанией Seldon. Сервис предоставляет доступ к актуальной информации из различных источников, включая официальные СМИ, интернет-издания и социальные медиа. Пользователи могут

фильтровать контент по ключевым словам, регионам и тематическим категориям, что делает сервис удобным для анализа новостей [3].

Одним из ключевых преимуществ Seldon News является интеграция с другими продуктами экосистемы Seldon, что позволяет получать доступ не только к новостям, но и к юридическим документам, пресс-релизам и отчетам. Сервис также предлагает базовые дашборды для отслеживания популярности событий, персон или компаний. Однако их функционал ограничен, что снижает возможности для глубокого анализа.

#### **1.2.4uCrawler**

uCrawler — это сервис для автоматического сбора и мониторинга новостей из различных источников. Основная функция сервиса заключается в извлечении новостного контента без необходимости ручной настройки. uCrawler собирает данные с сайтов на любом языке, включая заголовки, текст публикаций, даты и изображения [4].

Сервис использует алгоритмы машинного обучения для группировки статей по темам, что упрощает анализ событий и поиск взаимосвязей между ними. Пользователи могут настраивать фильтры по ключевым словам и получать доступ к архиву новостей, содержащему более 10 миллионов статей. Однако весь функционал доступен только в платных тарифах, а стоимость подписки зависит от количества отслеживаемых источников, что делает сервис дорогим для масштабной аналитической работы.

#### **1.2.5Flipboard**

Flipboard — это мобильное приложение, которое представляет новости в виде цифрового журнала. Сервис собирает статьи, блоги и другие материалы из различных источников и оформляет их в визуально привлекательные страницы. Пользователи могут настраивать ленту, выбирая интересующие темы и источники, а также создавать персональные "журналы" для сохранения и публикации статей [5].

Однако Flipboard имеет свои ограничения. Некоторые издания предоставляют доступ только к части статьи или требуют подписку для просмотра полного текста. Кроме того, сервис, хотя и собирает новости из российских источников, недоступен для пользователей из России.

### **1.2.6 Рамблер Новости**

Рамблер Новости — один из самых популярных российских новостных агрегаторов, предоставляющий доступ к актуальным событиям из более чем 100 источников. Сервис охватывает темы политики, экономики, спорта, технологий и шоу-бизнеса. Пользователи могут настраивать ленту новостей, выбирая интересные категории и темы [6].

Однако сервис имеет ряд недостатков. Новостная лента изобилует рекламными материалами, что отвлекает от чтения. Кроме того, новости из международных источников представлены ограниченно, что снижает привлекательность сервиса для тех, кто интересуется мировыми событиями.

### **1.2.7 Яндекс Дзен**

Яндекс Дзен — это социальная платформа с персонализированной лентой, формируемой на основе предпочтений пользователя. Платформа позволяет авторам создавать каналы и публиковать контент, а пользователям — подписываться на интересные темы и авторов. Новостная часть сервиса вынесена в раздел “Новости”, где каждая новость представлена в виде карточки с заголовком, краткой информацией и ссылками на источники [7].

Некоторые новости сопровождаются кратким пересказом, созданным нейросетью, однако таких материалов пока немного. Новости сгруппированы по темам, на которые можно подписываться. Однако функционал создания дайджестов или инструменты визуализации данных отсутствуют, что ограничивает возможности сервиса для аналитической работы.

По результатам сравнительного анализа можно составить таблицу (смотри таблица 1). Исходя из которой можно сказать, что умный корпус текстов СМИ является вполне конкурентноспособным для рассмотренного рынка. Также стоит учитывать тот факт, что проект разрабатывался с большим упором на предпочтения сотрудников Т-банка и на исследовательскую деятельность, а потому является нишевым в данной сфере.

Таблица 1 – Результаты сравнительного анализа

| Критерий/название                                                 | УКТСМИ | uCrawler | Рамблер<br>Новости | Flipboard | Smart<br>News | Feedly | Seldon.Ne<br>ws | Яндекс<br>Дзен |
|-------------------------------------------------------------------|--------|----------|--------------------|-----------|---------------|--------|-----------------|----------------|
| Персонализированные дайджесты                                     | +      | -        | -                  | +         | +             | +      | +               | + -            |
| Расширенная поисковая система                                     | +      | + -      | +                  | - +       | -             | + -    | + -             | +              |
| Персонализированные уведомления о появлении интересующих новостей | +      | -        | +                  | +         | +             | +      | -               | +              |
| Выгрузка новостей                                                 | +      | +        | -                  | -         | -             | +      | +               | -              |
| Работа с российскими СМИ                                          | +      | +        | +                  | +         | -             | +      | +               | +              |
| Автоматическая категоризация новостей                             | +      | + -      | +                  | +         | +             | +      | +               | +              |
| Настраиваемая статистика, дашборды                                | +      | -        | -                  | -         | -             | -      | +               | -              |
| Составление "выжимок" из статей                                   | +      | -        | -                  | -         | -             | +      | -               | +              |
| Мобильное приложение                                              | +      | -        | -                  | +         | +             | +      | -               | +              |
| Web версия                                                        | +      | +        | +                  | +         | -             | -      | +               | +              |
| Разметка статей                                                   | +      | -        | -                  | -         | -             | +      | -               | -              |
| Оставление заметок к статьям/дайджестам                           | +      | -        | -                  | -         | -             | +      | -               | -              |
| Добавление в избранное                                            | +      | -        | -                  | +         | -             | +      | -               | -              |

Продолжение таблицы 1

|     |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|
| API | - | + | - | - | - | + | - | - |
|-----|---|---|---|---|---|---|---|---|

### Выводы по главе

В данной главе представлено описание предметной области проекта и доказательство ее актуальности. Также приведен сравнительный анализ ближайших аналогов на рынке, выделены основные критерии и представлена таблица, показывающая недочеты и преимущества каждого из аналогов, благодаря которой доказывается потребность в разработке данного сервиса.

## **2 Проектирование системы, используемые технологии**

### **2.1 Функциональные требования к программе**

Программа должна реализовывать серверную часть веб-приложения следующего функционала:

- Авторизация и управление пользователями:
  - 1) Программа должна реализовывать авторизацию;
  - 2) Программа должна сохранять сессию пользователя в зависимости от типа входа (через мобильное/web приложение);
  - 3) Хранение данных пользователя в БД в зашифрованном формате;
  - 4) Разграничение функционала веб-приложения по ролям;
  - 5) Добавление новостных статей в избранное;
  - 6) Создание и подписка/отписка на новостные дайджесты;
  - 7) Функционал аннотирования статей;
  - 8) Функционал оставления заметок к статьям.
- Взаимодействие со статьями:
  - 1) Функционал получения статей и доп. информации о них, дальнейшее разбиение и сохранение в модели БД;
  - 2) Функционал сбора статей и запрос на формирование по ним дайджеста с последующей передачей на клиентскую часть веб-приложения;
  - 3) Функционал управления жизненным циклом статей (добавление/редактирование/удаление/чтение);
  - 4) Функционал выгрузки набора статей;
  - 5) Функционал фильтрации и поиска статей по заданным параметрам;
  - 6) Функционал оценивания классификации и краткого содержания статьи, а также оценка дайджестов.

### **2.2 Описание архитектуры приложения**

Как уже упоминалось ранее, данная работа является лишь частью умного корпуса текстов СМИ, общая схема которого представлена на рисунке (смотри рисунок 1).

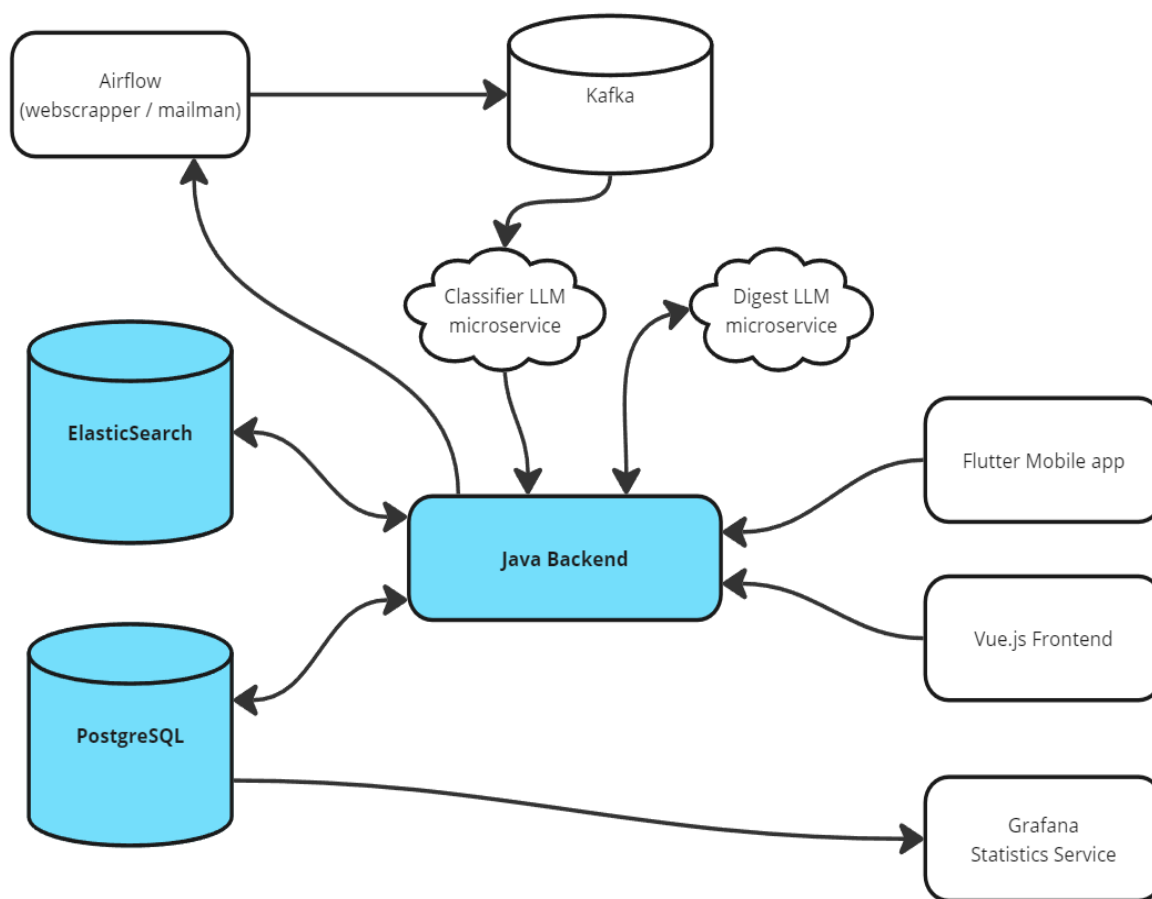


Рисунок 1 - Архитектура Системы

Как видно на диаграмме умный корпус текстов СМИ основан на микросервисной архитектуре, где синим цветом отмечены блоки, ответственность за реализацию которых лежит на данной ВКР. Подобная архитектура выбрана с учетом нескольких критериев:

- Проект является командным, где у каждого участника есть своя, четко разделенная зона ответственности, поэтому необходимо было обеспечить возможность ведения независимой друг от друга разработки, чтобы при внесении изменений в одну часть системы, не нарушалась работа остальных частей,
- Проект разрабатывался с учетом необходимости работы с большими объемами данных, которые нарастают ежедневно, что может вызвать проблемы с нагрузкой и стабильностью работы, потому важным критерием стала возможность масштабирования,
- Проект в основном разрабатывается для сотрудников банковской сферы, поэтому необходимо было уделить большое внимание отказоустойчивости системы.

Сервисы в системе взаимодействуют друг с другом через REST API, используя данные в формате JSON. Такое взаимодействие позволяет сервисам обмениваться информацией независимо от их внутренней реализации, что обеспечивает гибкость, масштабируемость и простоту интеграции.

### **2.3 Диаграмма прецедентов**

Для более детального рассмотрения работы системы поможет диаграмма прецедентов (смотри рисунок 2). Поскольку серверная часть не взаимодействует с пользователем напрямую, то акторами в данной диаграмме будут остальные сервисы умного корпуса текстов СМИ.

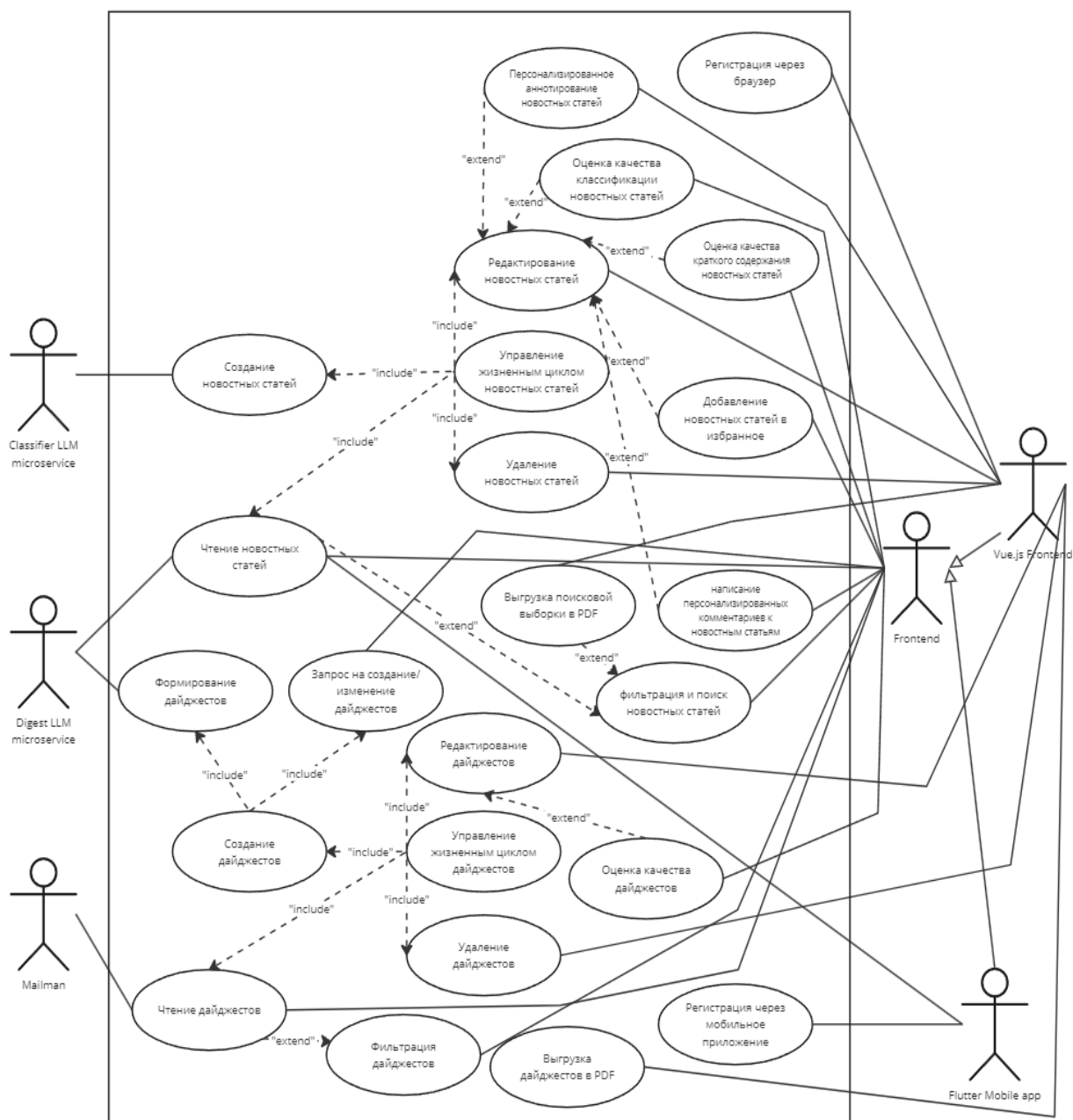


Рисунок 2 - Диаграмма прецедентов

На основе представленной диаграммы можно сделать вывод о том, с какой целью остальные сервисы взаимодействуют с серверной частью системы. Однако для полного понимания работы системы необходимо уточнить, что в ней реализовано разделение функциональных возможностей в зависимости от ролей пользователей. Всего предусмотрено две роли: администратор и обычный пользователь.

Администратор обладает полным доступом ко всему функционалу, описанному на диаграмме, включая возможность управления контентом. В то же время обычный пользователь ограничен в правах: ему недоступны операции редактирования и удаления

новостных статей. Такое ограничение введено с целью предотвращения потенциального повреждения критически важных данных, которое может возникнуть в случае несанкционированного или некорректного вмешательства в работу системы.

## **2.4 Архитектура решения**

Весь проект можно разделить на три составляющие:

- Corpus – включает в себя весь функционал, связанный с обработкой, хранением и управлением новостными статьями и дайджестами. Это ключевая часть системы, обеспечивающая работу с контентом,
- Admin – весь функционал, связанный с управлением пользователями и авторизацией,
- Config – содержит все необходимые настройки, требуемые для корректной работы приложения.

Также, на основе фреймворка Spring MVC, в Corpus и Admin присутствует разделение на уровни бизнес-логики и доступа к данным, предоставляя четкое разграничение архитектуры, что значительно упрощает разработку.

## **2.5 Выбор технологий реализации**

При разработке серверной части системы «Умный Корпус текстов СМИ» было принято решение использовать Java интерфейс Spring, вместе с Spring boot, предоставляющий автоматическую конфигурацию Spring приложений. Данный интерфейс полностью покрывает все потребности разработки, а также был выбран из-за большого опыта работы с ним.

## **2.6 Выбор базы данных**

При выборе базы данных для реализации данной работы необходимо было учитывать два ключевых фактора. Во-первых, наличие сильной связи между сущностями, особенно с новостными статьями, что предполагает использование реляционных баз данных. Во-вторых, необходимость выполнения полнотекстового поиска по большим массивам текстовых данных, что требует применения нереляционной базы данных, такой как Elasticsearch, которая обеспечивает высокую скорость выполнения подобных запросов. Однако было принято решение использовать бесплатный, доступный в России аналог - OpenSearch, который не уступает эффективности Elasticsearch.

Для реализации описанных требований было принято решение разделить сущность новостной статьи на две отдельные части: DocMeta и DocES. В DocMeta хранится вся метainформация о документах, которая может быть использована для фильтрации и

взаимодействия со связными сущностями БД. В DocES содержится информация, необходимая для выполнения ресурсоёмких операций контекстного поиска, включая заголовок и основное содержание новостной статьи. Для хранения данных в реляционной базе данных был выбран PostgreSQL, так как он полностью соответствует требованиям проекта, а также имеется большой опыт в работе с ним.

Процесс создания новостной статьи в системе можно описать следующим алгоритмом:

- Система классификации и сбора новостных статей отправляет запрос на создание новой статьи, передавая в теле запроса необходимый набор данных,
- Данные разделяются на две группы в соответствии с сущностями DocMeta и DocES,
- На первом этапе сохраняется сущность DocMeta в PostgreSQL. Если сохранение завершается неудачей, процесс прерывается, и система возвращает ошибку. В случае успешного сохранения генерируется уникальный идентификатор статьи, который присваивается сущности DocES,
- На втором этапе сущность DocES сохраняется в OpenSearch. Если сохранение не удаётся, процесс завершается с возвратом ошибки.

Таким образом, каждая новостная статья представлена двумя сущностями, хранящимися в разных базах данных, но связанными общим идентификатором. Такой подход обеспечивает эффективное управление данными и высокую производительность при выполнении операций поиска и фильтрации.

## 2.7 Архитектура базы данных в целом и особенности хранения дайджестов и подписок на них

Полную архитектуру БД можно понять по схеме (смотри рисунок 3).

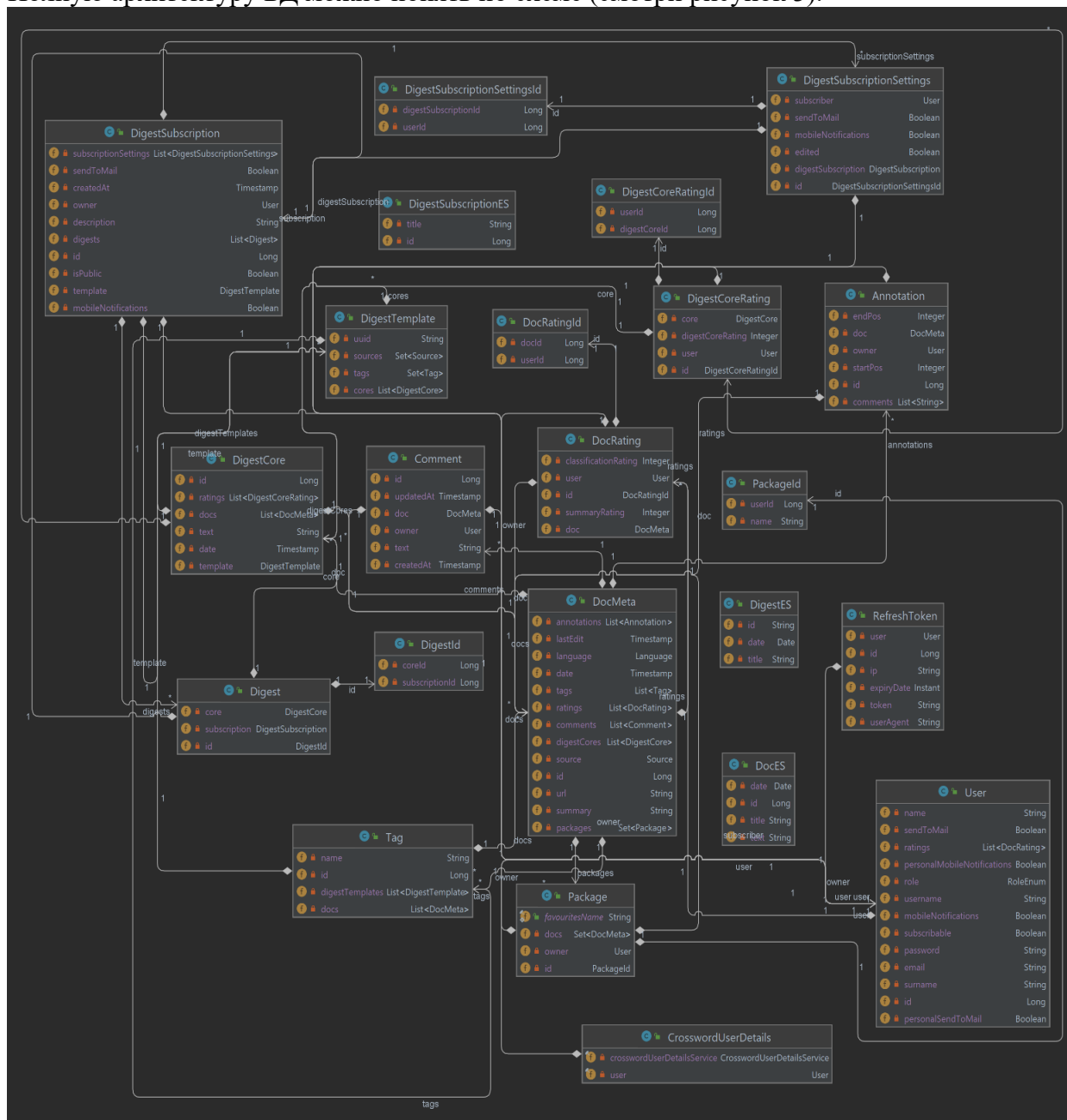


Рисунок 3 - Архитектура БД

Однако следует отдельно отметить способ хранения в БД новостных статей, дайджестов и подписок на дайджесты. Особенности, связанные с хранением новостных статей, рассматривались в предыдущем пункте, поэтому сейчас рассмотрим оставшиеся.

Все остальные сервисы общаются с новостными дайджестами и подписками на них, однако для оптимизации и производительности, дайджесты и подписки разделены на следующие сущности:

- Дайджест:
  - 1) Ядро дайджеста;
  - 2) Шаблон дайджеста;
  - 3) Рейтинг дайджеста;
- Подписка на дайджест:
  - 1) Подписка на дайджест;
  - 2) Настройки подписки на дайджест.

По итогу сам дайджест, который получает пользователь, является сочетанием всех перечисленных моделей, чтобы упростить понимание, можно воспользоваться схемой (смотри рисунок 4).

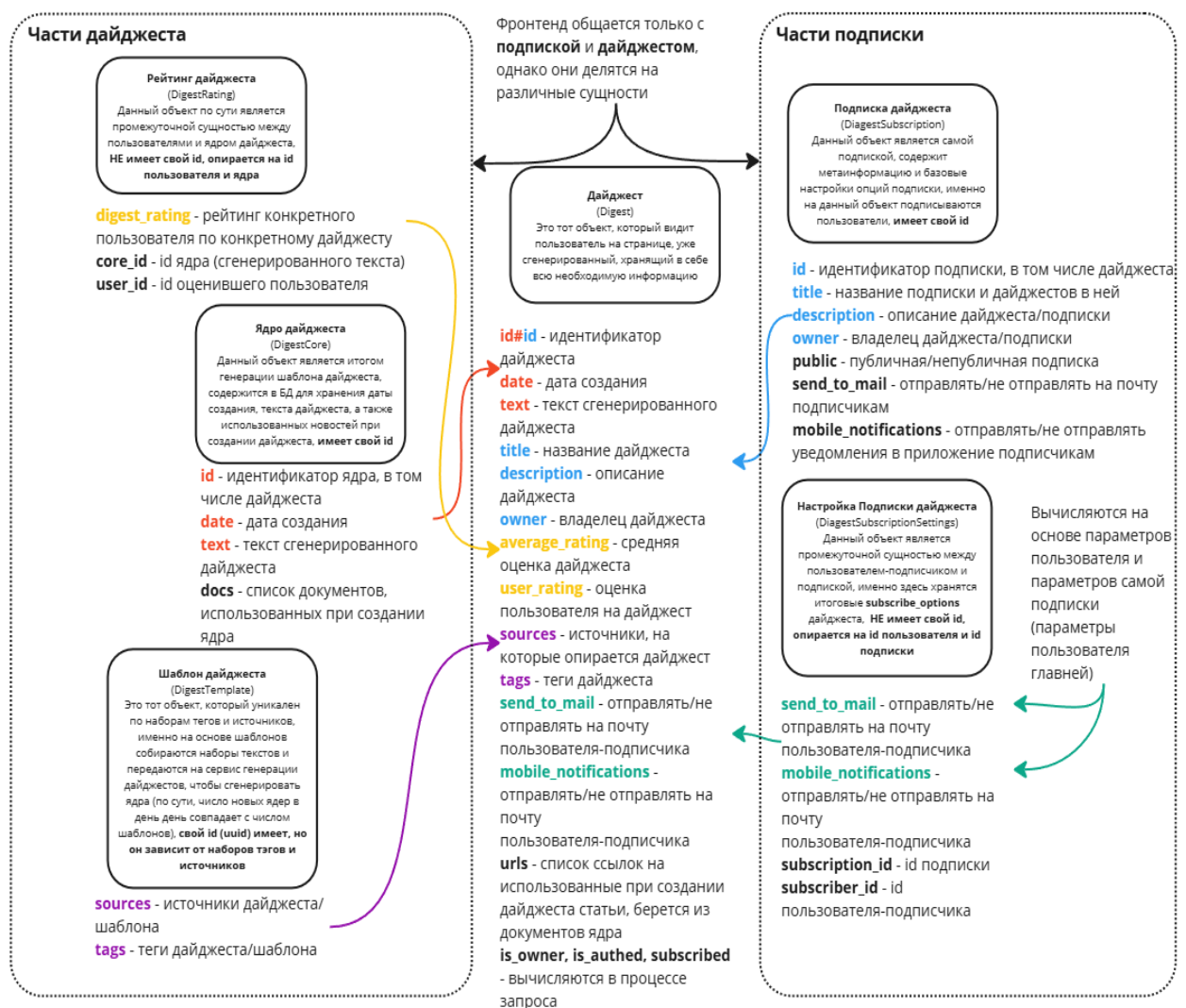


Рисунок 4 - Схема создания дайджеста

### **Выводы по главе**

В данной главе были рассмотрены функциональные требования, архитектура веб-приложения в целом, также взаимодействие других систем с серверной частью умного корпуса СМИ, выбор технологий реализации, а также выбор реализации базы данных проекта и особенности хранения некоторых из моделей в ней.

### **3 Программная реализация системы**

Данный проект является промежуточным звеном между базами данных и остальными частями Умного корпуса текстов СМИ, которое принимает только REST запросы, а также формирует ответы на них, пользовательский интерфейс не предусмотрен и не требуется. По этой причине, чтобы показать работоспособность реализованного функционала, будет использоваться Postman, способный симитировать требуемые REST запросы.

К каждому функционалу будет представлено краткое описание, входные данные, а также возможные результаты выполнения, зависящие от них. Однако не весь функционал зависит только от работы реализованного сервиса, так, например, при создании новостного дайджеста, необходимо собрать набор документов по заданным фильмам, а затем передать их на сервис формирования дайджестов, реализация которого не входит в данный проект, поэтому необходимо считать, что о работе остальных сервисов известны только ожидаемые параметры на вход, а также на выход.

#### **3.1 Функционал, связанный с управлением пользователями и авторизацией**

Для упрощения понимания работы данного функционала, предварительно будут описаны существующие схемы, которые могут понадобиться для выполнения запросов, в которых также будут представлено описание всех присущих полей.

- Схема регистрации пользователя:
  - 1) “name” – имя регистрируемого пользователя;
  - 2) “surname” - фамилия регистрируемого пользователя;
  - 3) “username” - логин регистрируемого пользователя;
  - 4) “password” - пароль регистрируемого пользователя;
  - 5) “email” - почта регистрируемого пользователя;
- Схема авторизации пользователя:
  - 1) “username” – почта или логин авторизируемого пользователя;
  - 2) “password” – пароль авторизируемого пользователя;
- Схема набора разрешений:
  - 1) “authorities” – список разрешений пользователей.

### 3.1.1 Регистрация пользователя

Регистрация пользователя позволяет создавать новых пользователей, которые смогут воспользоваться своими учетными данными, чтобы получить доступ к функционалу программы, в соответствии со своими правами. Чтобы зарегистрироваться, пользователю необходимо воспользоваться запросом - POST “/users/register”, а также передать в тело данного запроса данные, согласно схеме регистрации пользователя.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – пользователь успешно зарегистрирован (смотри рисунок 5),
- 400 – введенные “username” – логин пользователя или “email” – почта пользователя, уже существуют в базе данных и присвоены другим пользователям (смотри рисунок 6).

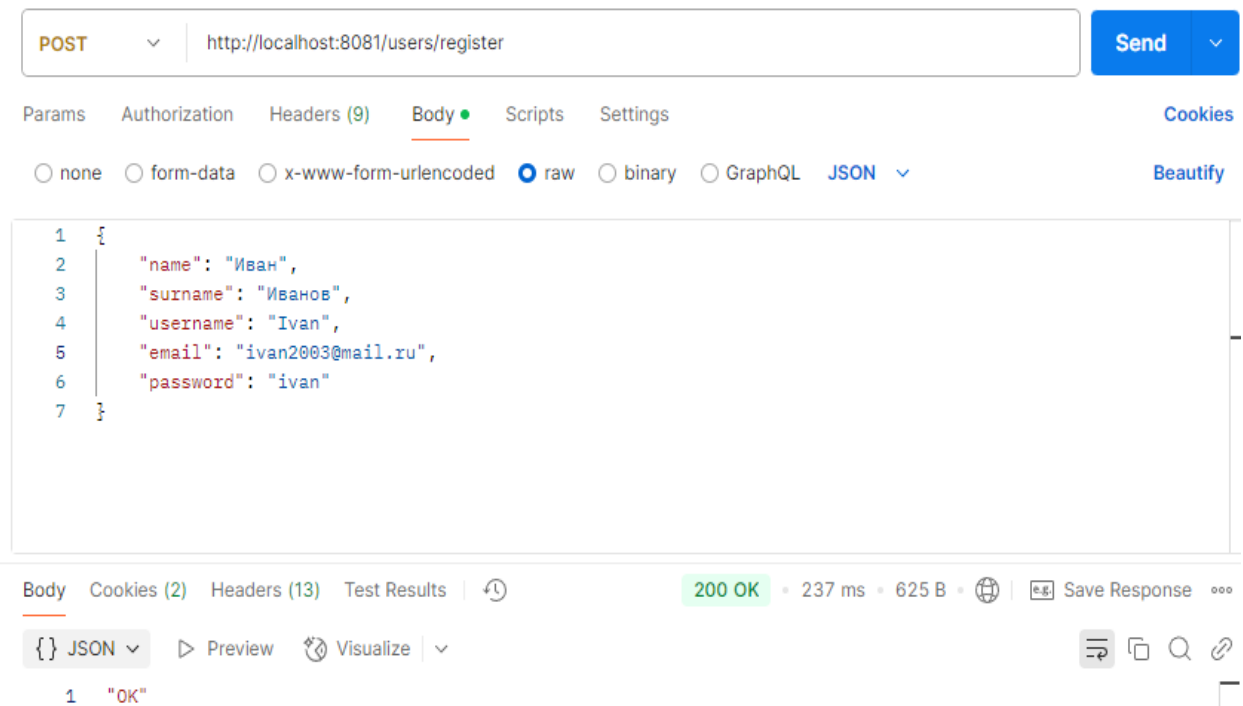


Рисунок 5 - Пример запроса регистрации, ответ 200

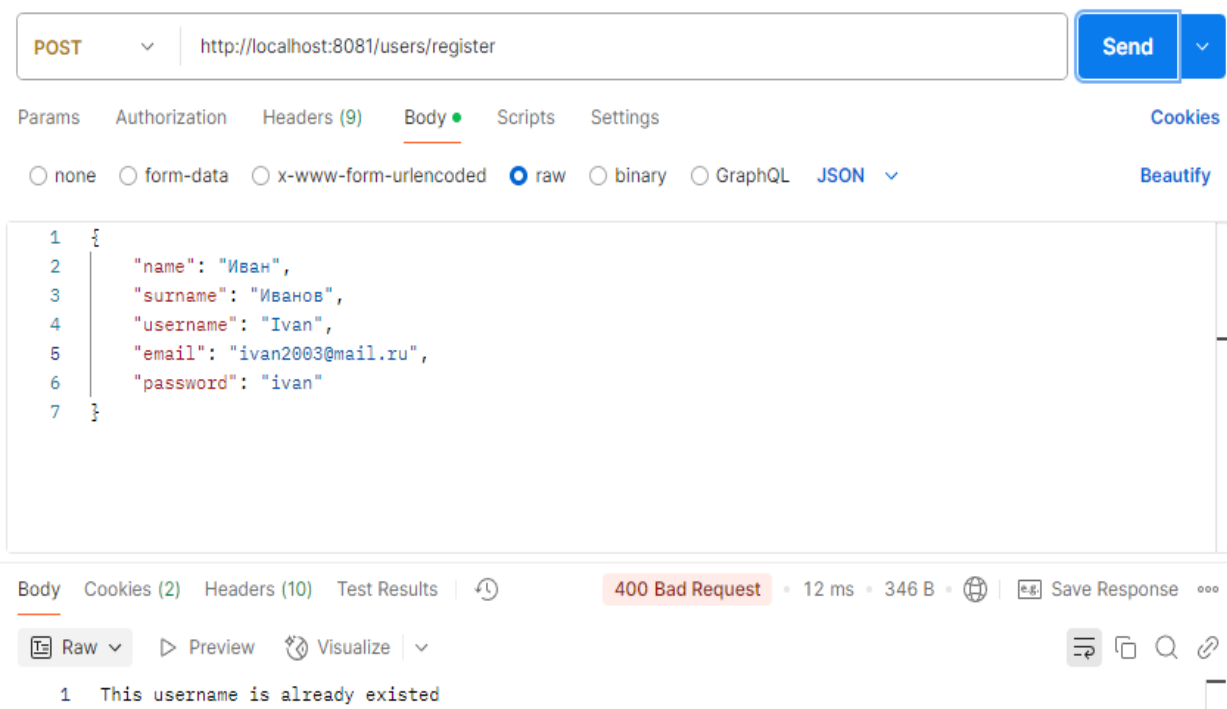


Рисунок 6 - Пример запроса регистрации, ответ 400

После успешной регистрации пользователя:

- Пользователю присваивается соответствующая роль, если его логин или почта присутствуют в списках ролей,
- Пользователю выдается идентификационный номер и сохраняется в БД с соответствующей информацией о себе. Стоит также отметить, что пароль сохраняется только в зашифрованном виде, поэтому доступа к нему нет даже у программы,
- Для пользователя создается персональная папка “Избранное”,
- Создаются access и refresh токены, которые записываются в https only куки и автоматически передаются во все последующие запросы.

### 3.1.2 Авторизация пользователя

Авторизация пользователя позволяет пользователям получить доступ к функционалу программы, в соответствии с их правами доступа, если те уже были зарегистрированы и ввели корректные соответствующие данные. Чтобы проверить совпадение зашифрованного пароля, хранящегося в БД и переданного в запросе, переданный пароль также шифруется и в итоге проверяется с хранящимся, если они совпали, то и дешифрованные версии равны, в связи с особенностями функции шифрования. Чтобы авторизовать пользователя, необходимо воспользоваться запросом – POST “/users/login”, а в тело запроса предать данные, соответствующие схеме авторизации пользователя.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – пользователь успешно авторизован (смотри рисунок 7),
- 400 – введены некорректные почта/логин или пароль пользователя (смотри рисунок 8).

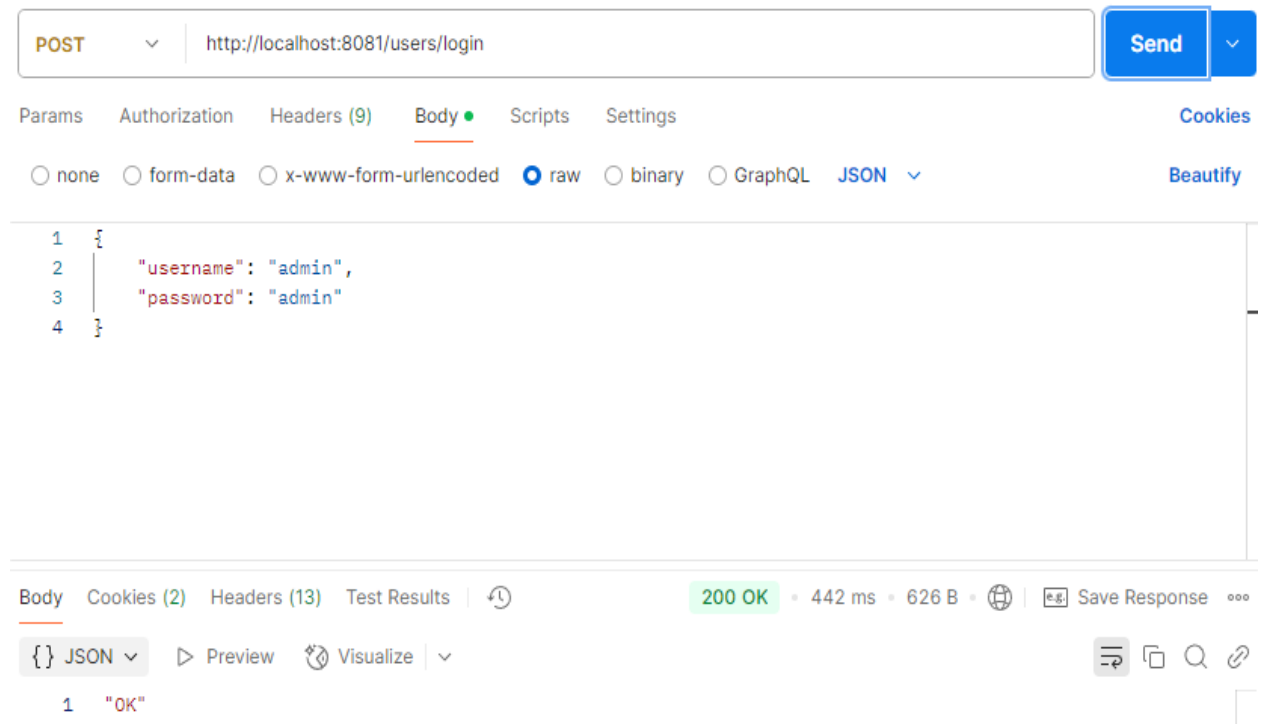


Рисунок 7 - Пример запроса авторизации, ответ 200

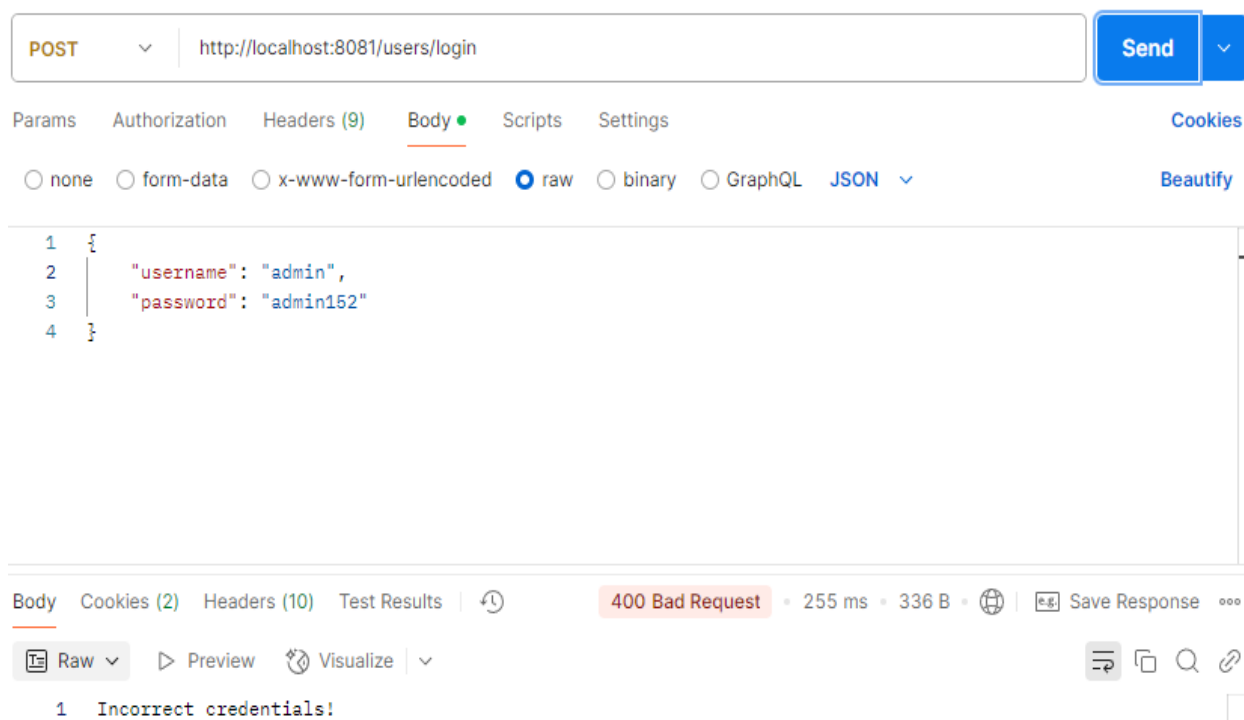


Рисунок 8 - Пример запроса авторизации, ответ 400

После успешной авторизации пользователя происходят те же события, что и при регистрации, за исключением того, что новый пользователь и данные о нем не создаются и не сохраняются в базу данных.

### 3.1.3 Проверка разрешений пользователя

Некоторый функционал доступен только для пользователей с определенными правами доступа, что справедливо не только для серверной части, но и для остальных частей Умного корпуса СМИ, чтобы узнать разрешения пользователя, необходимо воспользоваться запросом – GET “/users/check\_auth”, данные о пользователе берутся из access токена.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – у пользователя успешно получилось достать разрешения, они будут переданы в теле ответа в формате схемы набора разрешений (смотри рисунок 9),
- 401 – пользователь не авторизован, а данный функционал требует авторизации (смотри рисунок 10).

GET http://localhost:8081/users/check\_auth Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     |           |

---

Body Cookies (2) Headers (11) Test Results 200 OK 7 ms 370 B Save Response

{ } JSON Preview Visualize

```

1 {
2   "authorities": [
3     "edit_delete_docs"
4   ]
5 }

```

Рисунок 9 - Пример запроса проверки разрешений пользователя, ответ 200

GET http://localhost:8081/users/check\_auth Send

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     |           |

---

Body Cookies (2) Headers (11) Test Results 401 Unauthorized 24 ms 452 B Save Response

{ } JSON Preview Visualize

```

1 {
2   "timestamp": "2025-03-18T00:20:09.789+00:00",
3   "status": 401,
4   "error": "Unauthorized",
5   "path": "/users/check_auth"
6 }

```

Рисунок 10 - Пример запроса проверки разрешений пользователя, ответ 401

### **3.2 Функционал, связанный с обработкой, хранением и управлением новостными статьями**

Как и в функционале, связанном с управлением пользователями и авторизацией, представим схемы запросов и ответов с описанием соответствующих им полей:

– Схема создания новостной статьи (другое название новостной статьи - документ):

- 1) “summary” – краткое содержание статьи;
- 2) “date” – дата создания статьи;
- 3) “language” – язык, на котором написана статья;
- 4) “text” – текст статьи;
- 5) “title” – название статьи;
- 6) “URL” – адрес статьи;
- 7) “source” – название источника, с которого взята статья;
- 8) “tags” – список тэгов, присвоенных статье;

– Схема получения документа:

- 1) “id” – идентификатор документа;
  - 2) “title” – название документа;
  - 3) “summary” – краткое содержание документа;
  - 4) “text” – текст документа;
  - 5) “date” – дата создания документа;
  - 6) “favourite” – флаг, оповещающий о том, добавил ли пользователь данный документ в папку “Избранное”;
  - 7) “language” – язык, на котором написан документ;
  - 8) “source” – источник, из которого взят документ;
  - 9) “tags” – список тэгов, соответствующих документу;
  - 10) “last\_edit” – дата последнего изменения документа;
  - 11) “rating\_summary” – оценка пользователя краткого содержания документа;
  - 12) “rating\_classification” – оценка пользователя классификации документа;
  - 13) “is\_authed” – флаг, оповещающий о том, что пользователь, выполнивший запрос – не авторизирован;
  - 14) “URL” – адрес документа;
- Схема редактирования документа:
- 1) “summary” – краткое содержание статьи;
  - 2) “date” – дата создания статьи;

- 3) “language” – язык, на котором написана статья;
  - 4) “text” – текст статьи;
  - 5) “title” – название статьи;
  - 6) “URL” – адрес статьи;
  - 7) “source” – название источника, с которого взята статья;
  - 8) “tags” – список тэгов, присвоенных статье;
  - 9) drop\_rating\_summary – флаг, определяющие, необходимо ли сбросить рейтинг краткой выжимки документа;
  - 10) drop\_rating\_classification – флаг, определяющий, необходимо ли сбросить рейтинг классификации документа;
- Схема оценки документа:
- 1) “summary\_rating” – рейтинг краткой выжимки документа;
  - 2) “classification\_rating” – рейтинг классификации документа;
- Схема поиска документа:
- 1) “id” – идентификатор документа, для поиска по id;
  - 2) “language” – список возможных языков, на которых могут быть написаны искомые документы, параметр фильтрации;
  - 3) “sources” - список возможных источников, с которых могли прийти искомые документы, параметр фильтрации;
  - 4) “tags” - список возможных тегов, которые могут быть присвоены искомым документам, параметр фильтрации;
  - 5) “search\_body” – поисковой запрос, используемый в точном и в синтаксическом поисках;
  - 6) “search\_mode” – id/certain/semantic определяющее тип поиска поле;
  - 7) “date\_from” – дата, от которой могли быть созданы документы, параметр фильтрации;
  - 8) “date\_to” – дата, до которой могли быть созданы документы, параметр фильтрации;
  - 9) “page\_number” – номер страницы, которую необходимо вернуть;
  - 10) “matches\_per\_page” – число результатов поиска, отображаемы на одной странице;
- Схема создания аннотации:
- 1) “start” – позиция начала аннотации относительно всего текста документа;
  - 2) “end” – позиция конца аннотации относительно всего текста документа;
  - 3) “comments” – список комментариев, предоставляемых к аннотации;
- Схема создания комментария к документу:
- 1) “text” – текст создаваемого комментария;

2) “created\_at” – дата создания комментария.

### 3.2.1 Создание нового документа

Данный функционал позволяет создавать новые новостные статьи, являющиеся неотъемлемой частью всей системы, на которые опирается большинство функционала. Чтобы создать статью, необходимо воспользоваться запросом – POST “/documents/create”, передав в тело запроса данные в соответствии со схемой создания новостной статьи.

После отправления запроса, единственным возможным вариантом ответа будет 200, поскольку у документа нет чувствительных данных, которые необходимо проверять на уникальность (смотри рисунок 11).

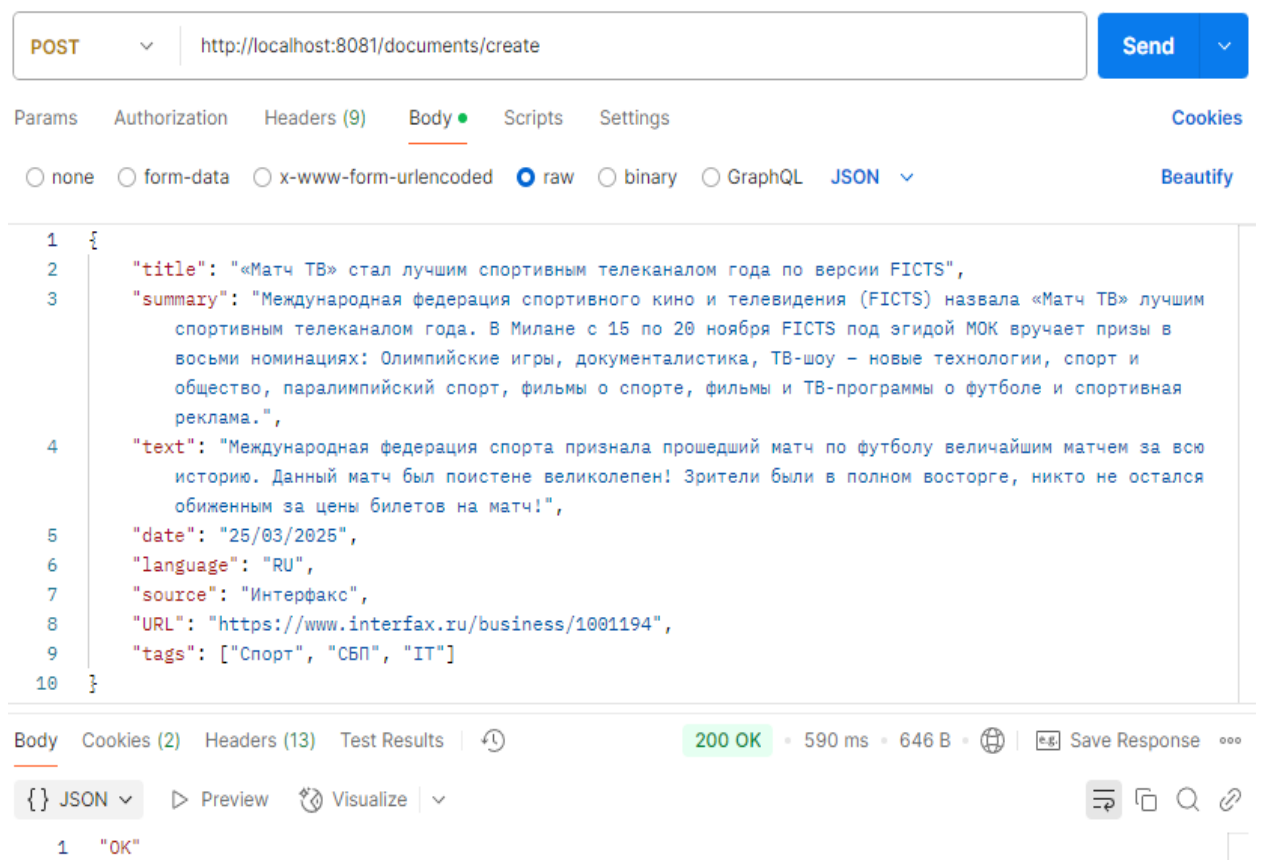


Рисунок 11 - Пример запроса создания документа, ответ 200

### 3.2.2 Получение списка всех документов

Данный функционал позволяет получить весь список документов, который хранится в базе данных. Чтобы получить список, необходимо выполнить запрос – GET “/documents”. Данные о пользователе получаются из access токена.

После отправки запроса, единственный возможный вариант ответа – 200, поскольку у данного функционала не предназначена какая либо валидация данных, за исключением той, что сверяет синхронизацию баз данных PostgreSQL и OpenSearch,

провал которой приводит к ошибке 500, однако это возможно только в случае некорректной реализации какого-либо из функционала. Список документов возвращается теле ответа, в виде списка объектов, соответствующих схеме получения документа. Также стоит учесть, что поля, связанные с пользователем могут иметь значения null, кроме “is\_authed”, что будет означать невозможность получить данные из БД, поскольку пользователь не авторизирован или данные соответствующих полей отсутствуют (смотри рисунок 12 и рисунок 13).

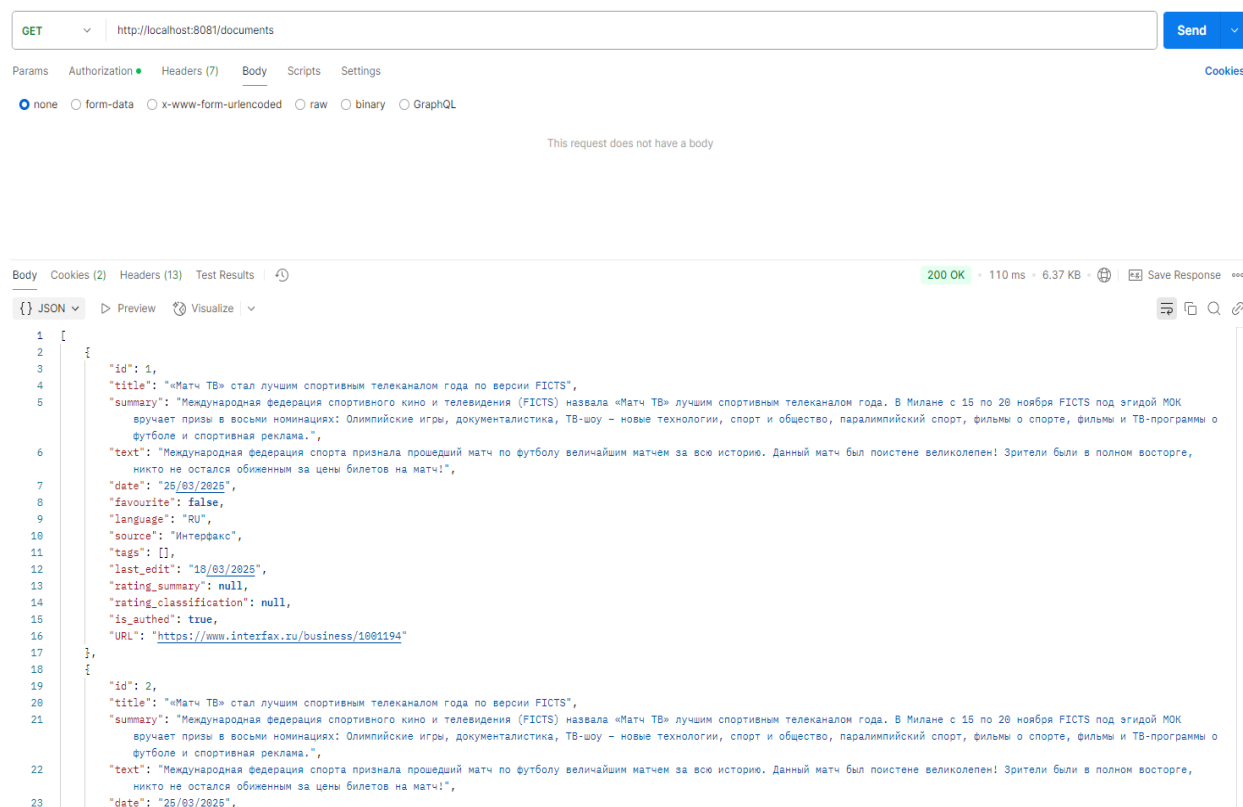


Рисунок 12 - Пример запроса получения списка всех документов, ответ 200, пользователь авторизован

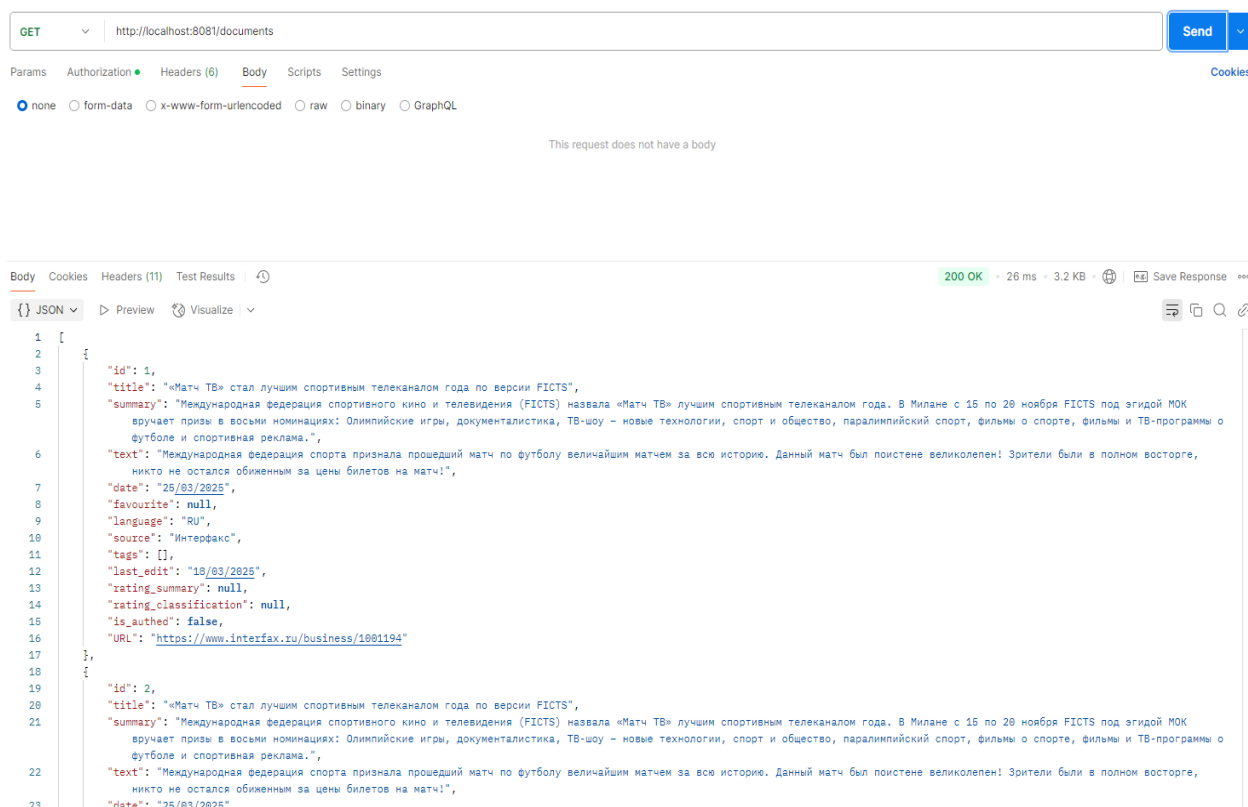


Рисунок 13 - Пример запроса получения списка всех документов, ответ 200, пользователь не авторизован

### 3.2.3 Получение документа по id

Данный функционал, в отличие от получения всех документов, предназначен для получения одного конкретного документа. Чтобы получить документ по `id`, необходимо выполнить запрос – GET `“/documents/{id}”`, где `id` – идентификатор искомого документа.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось предоставить документ по `id` (смотри рисунок 14),
- 404 – документа с подобным `id` не существует в базе данных (смотри рисунок 15).

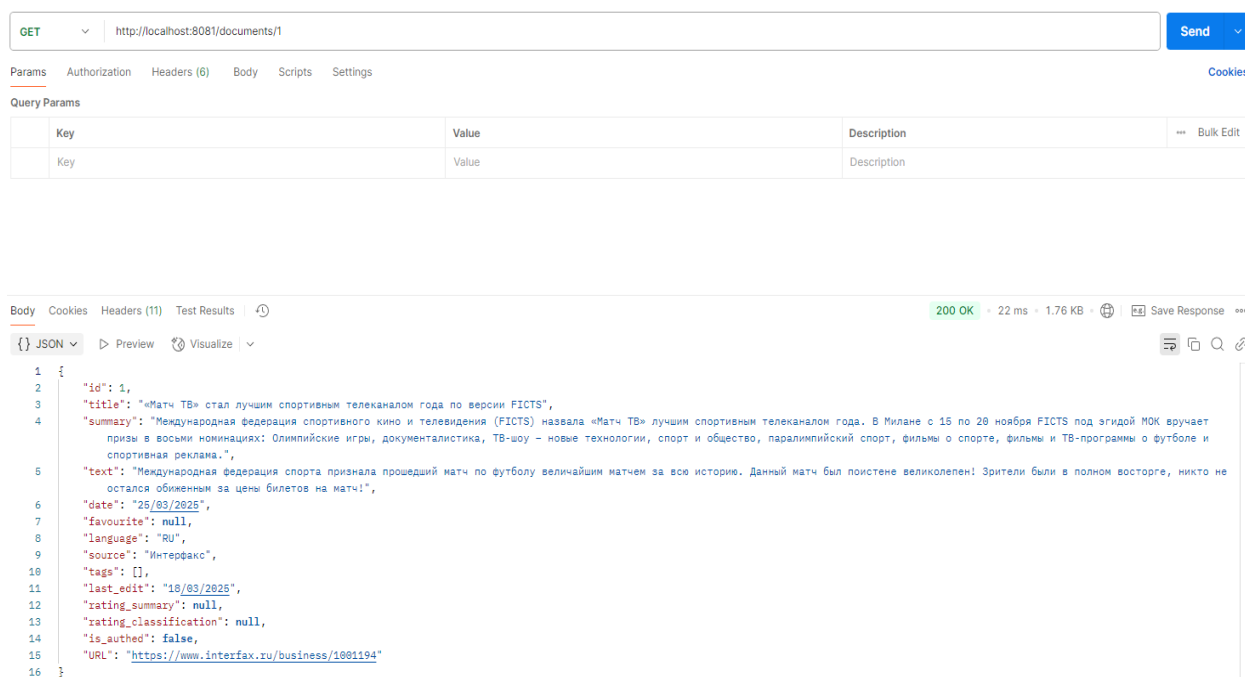


Рисунок 14 - Пример запроса получения документа по id, ответ 200

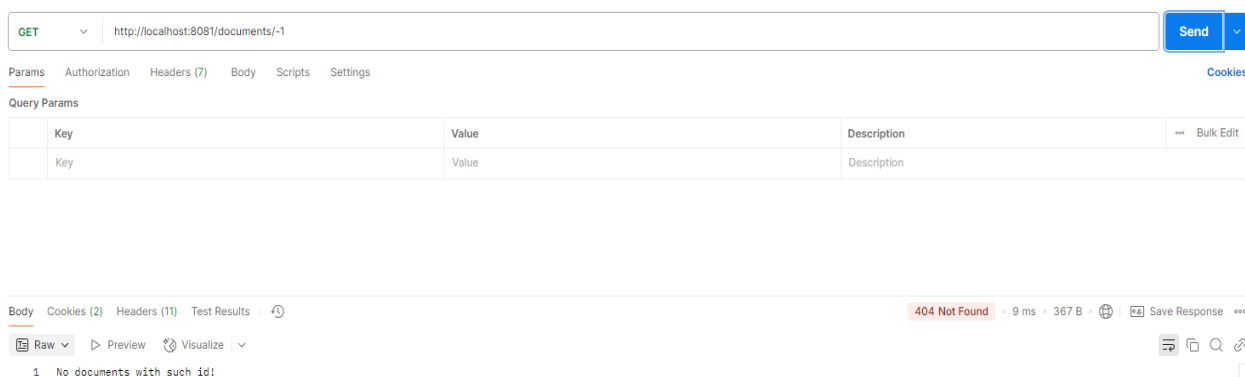


Рисунок 15- Пример запроса получения документа по id, ответ 404

После успешного получения документа, в тело ответа передается один объект, соответствующий схеме получения документа. Все особенности получения данных документа, связанных с пользователем, идентичны особенностям получения списка документов.

### 3.2.4 Редактирование документа по id

Данным функционалом необходимо пользоваться очень аккуратно, потому что изменение текста или краткой выжимки приведет к очень большому количеству потери данных, по этой причине доступ к данному функционалу имеют пользователи с отдельным разрешением -“EDIT\_DELETE\_DOCS”. Чтобы изменить документ, необходимо воспользоваться запросом – PUT “/documents/{id}/edit”, где id –

идентификатор документа, с которым производят изменения, а в тело запроса передаются данные в соответствии со схемой изменения документа. Также стоит обратить внимание, что запрос вида PUT, который требует передачи не только тех полей, которые необходимо изменить, но и всех остальных, иначе их отсутствие будет рассчитываться, как причина для присваивания им значения null.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось изменить документ (смотри рисунок 16),
- 401 – попытка изменить документ, не авторизовавшись, что запрещено (смотри рисунок 17),
- 403 – попытка изменить документ, не обладая необходимыми правами доступа (“EDIT\_DELETE\_DOCS”) (смотри рисунок 18),
- 404 – попытка изменить документ с id документа, которого нет в базе данных (смотри рисунок 19).

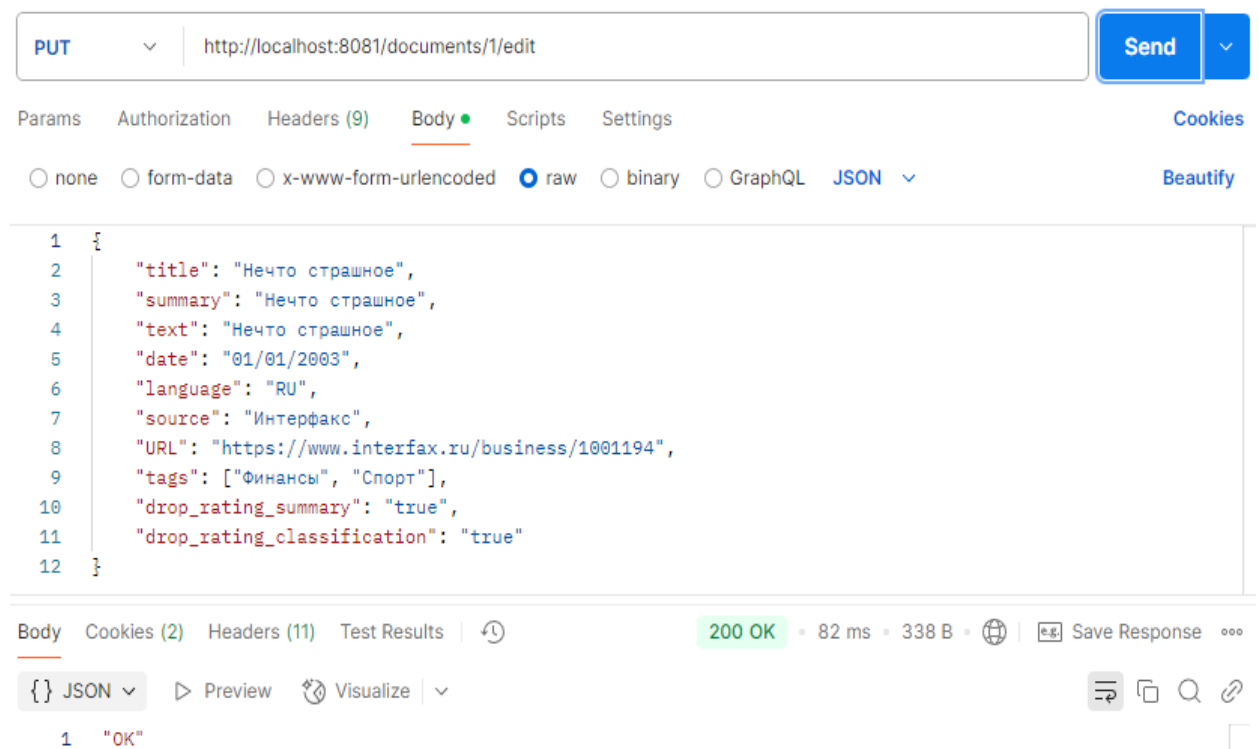


Рисунок 16 - Пример запроса редактирования документа по id, ответ 200

PUT http://localhost:8081/documents/1/edit Send

Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Beautify

```
1 {
2   "title": "Нечто страшное",
3   "summary": "Нечто страшное",
4   "text": "Нечто страшное",
5   "date": "01/01/2003",
6   "language": "RU",
7   "source": "Интерфакс",
8   "URL": "https://www.interfax.ru/business/1001194",
9   "tags": ["Финансы", "Спорт"],
10  "drop_rating_summary": "true",
11  "drop_rating_classification": "true"
12 }
```

Body Cookies Headers (11) Test Results 401 Unauthorized 6 ms 452 B Save Response

{ JSON Preview Visualize

```
1 {
2   "timestamp": "2025-03-18T03:20:37.635+00:00",
3   "status": 401,
4   "error": "Unauthorized",
5   "path": "/documents/1/edit"
6 }
```

Рисунок 17 - Пример запроса редактирования документа по id, ответ 401

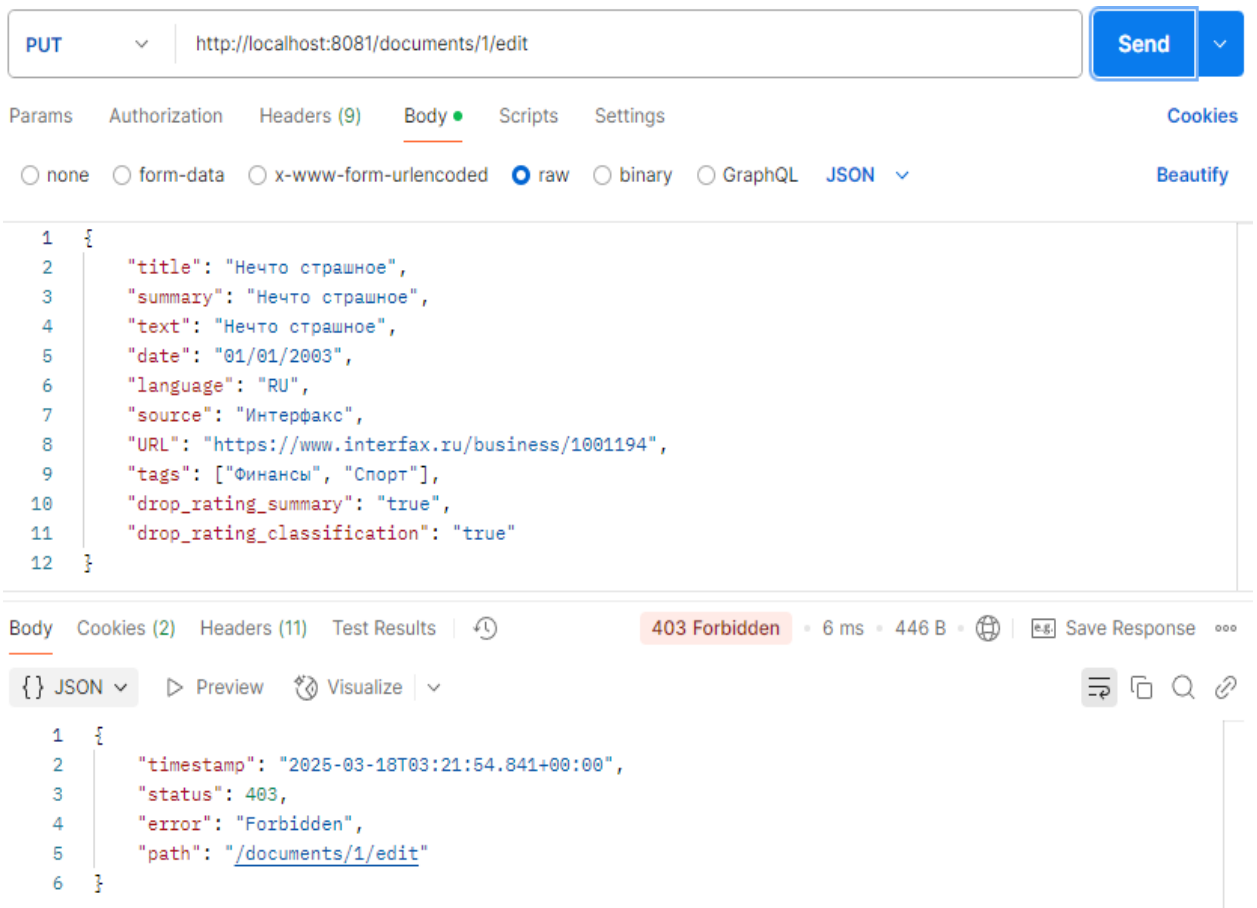


Рисунок 18 - Пример запроса редактирования документа по id, ответ 403

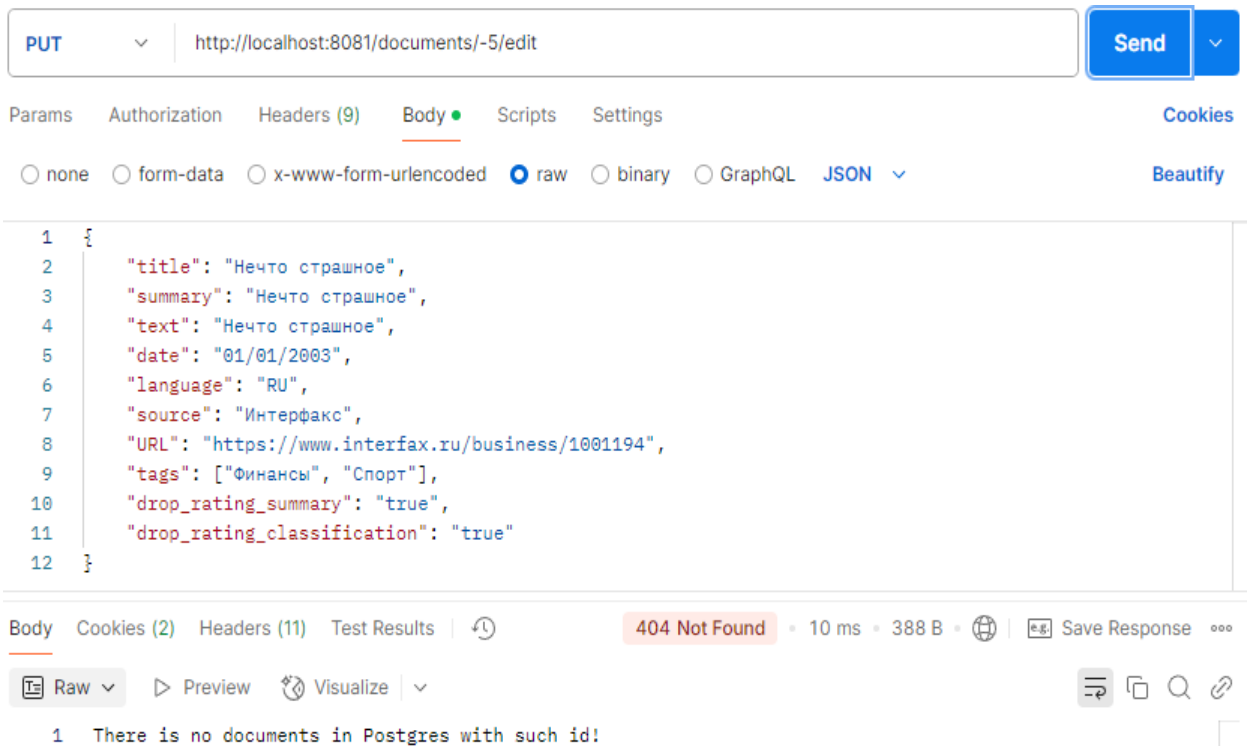


Рисунок 19 - Пример запроса редактирования документа по id, ответ 404

### 3.2.5 Удаление документа по id

С данным функционалом необходимо быть еще аккуратнее, чем с изменением документа, потому что в данном случае нет даже варианта выполнить запрос, чтобы не потерять множество данных, поэтому для удаления также необходимо иметь пользовательское разрешение “EDIT\_DELETE\_DOCS”. Чтобы удалить документ, необходимо выполнить запрос – DELETE “/documents/{id}”, где id – идентификатор удаляемого документа.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить документ (смотри рисунок 20),
- 401 – попытка удалить документ, не авторизовавшись, что запрещено (смотри рисунок 21),
- 403 – попытка удалить документ, не обладая необходимыми правами доступа (“EDIT\_DELETE\_DOCS”) (смотри рисунок 22),
- 404 – попытка удалить документ с id документа, которого нет в базе данных (смотри рисунок 23).

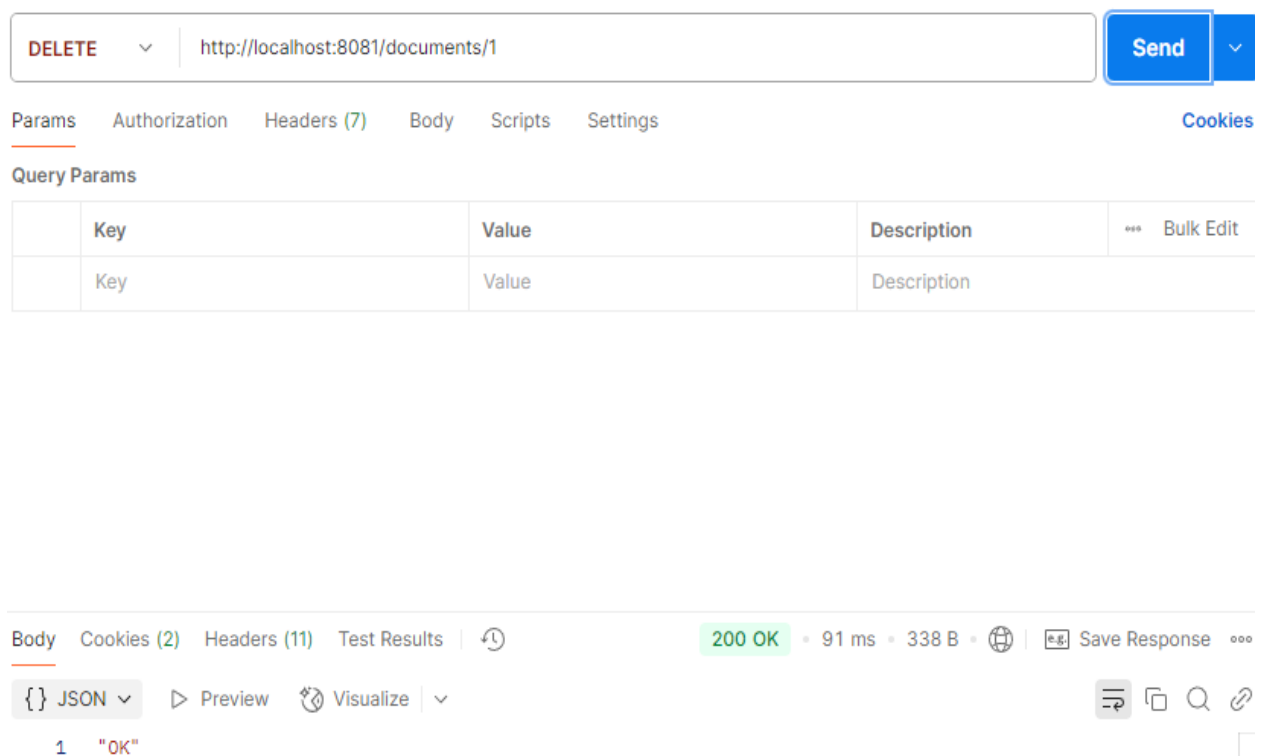


Рисунок 20 - Пример запроса удаления документа по id, ответ 200

DELETE    http://localhost:8081/documents/1    Send

Params    Authorization    Headers (6)    Body    Scripts    Settings    Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     |           |

Body    Cookies    Headers (11)    Test Results    401 Unauthorized    4 ms    447 B    Save Response

{ } JSON    Preview    Visualize

```
1 {
2   "timestamp": "2025-03-18T03:31:45.243+00:00",
3   "status": 401,
4   "error": "Unauthorized",
5   "path": "/documents/1"
6 }
```

Рисунок 21 - Пример запроса удаления документа по id, ответ 401

DELETE ▼ http://localhost:8081/documents/1 Send ▼

Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

|  | Key | Value | Description | ... | Bulk Edit |
|--|-----|-------|-------------|-----|-----------|
|  | Key | Value | Description |     | Inf       |

Body Cookies (2) Headers (11) Test Results 🕒 403 Forbidden • 6 ms • 441 B • 🌐 💾 Save Response ...

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ ☰ 📄 🔍 🔗

```

1  {
2    "timestamp": "2025-03-18T03:32:54.906+00:00",
3    "status": 403,
4    "error": "Forbidden",
5    "path": "/documents/1"

```

Рисунок 22 - Пример запроса удаления документа по id, ответ 403

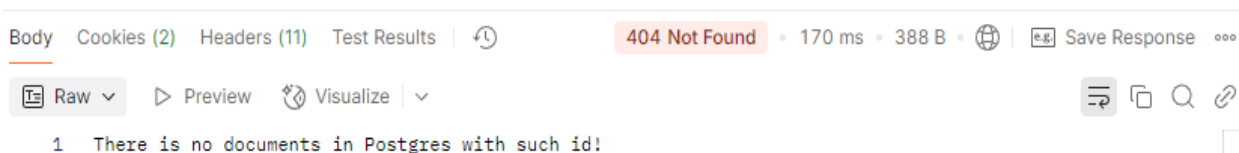
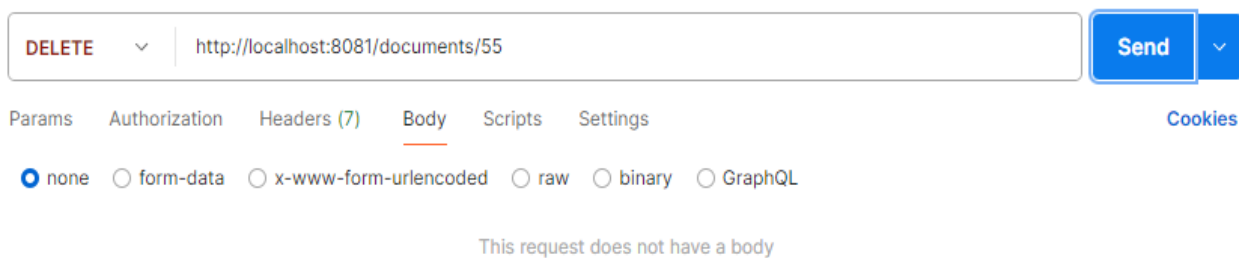


Рисунок 23 - Пример запроса удаления документа по id, ответ 404

### 3.2.6 Оценка документа

Данный функционал позволяет за один запрос сразу установить рейтинг и классификации, и краткой выжимки. Для оценки документа необходимо выполнить запрос – PATCH “/documents/{id}/rate”, где id – идентификатор оцениваемого документа, соответствующие рейтинги передаются в тело запроса, согласно схеме оценки документа, а информация о пользователе берется из access токена. Также стоит заметить, что запрос PATCH, значит не требует обязательного заполнения всех полей, если те не должны быть изменены, их значения в таком случае останутся не тронутыми.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось оценить документ (смотри рисунок 24),
- 401 – попытка оценить документ, не авторизовавшись, что запрещено (смотри рисунок 25),
- 400 – попытка оценить документ, при этом параметры оценки выходят за границы значений от 1 до 5, что запрещено (смотри рисунок 26),
- 404 – попытка оценить документ по id документа, которого нет в базе данных (смотри рисунок 27).

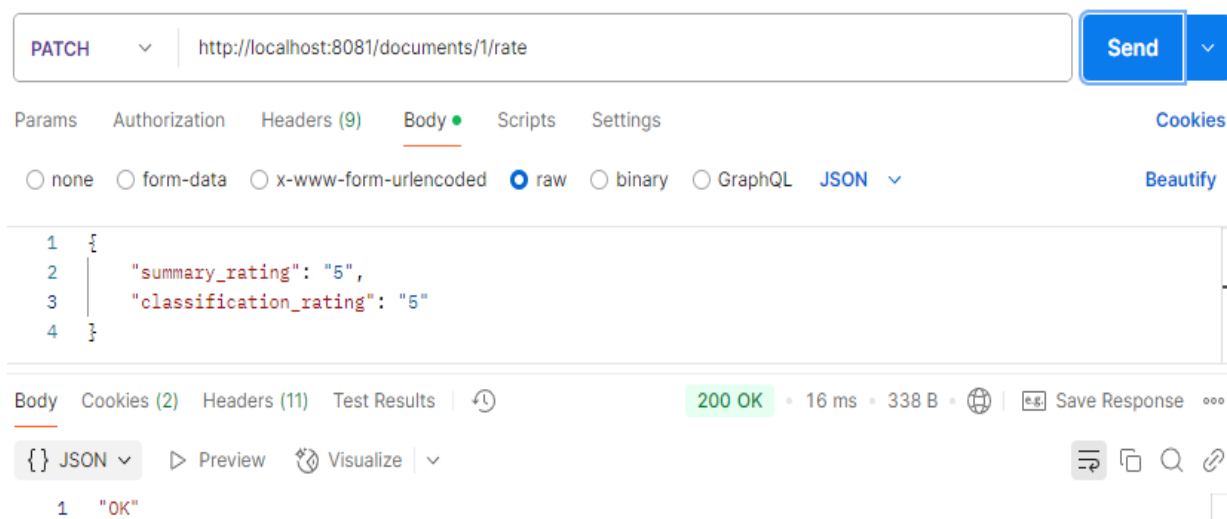


Рисунок 24 - Пример запроса оценки документа по id, ответ 200

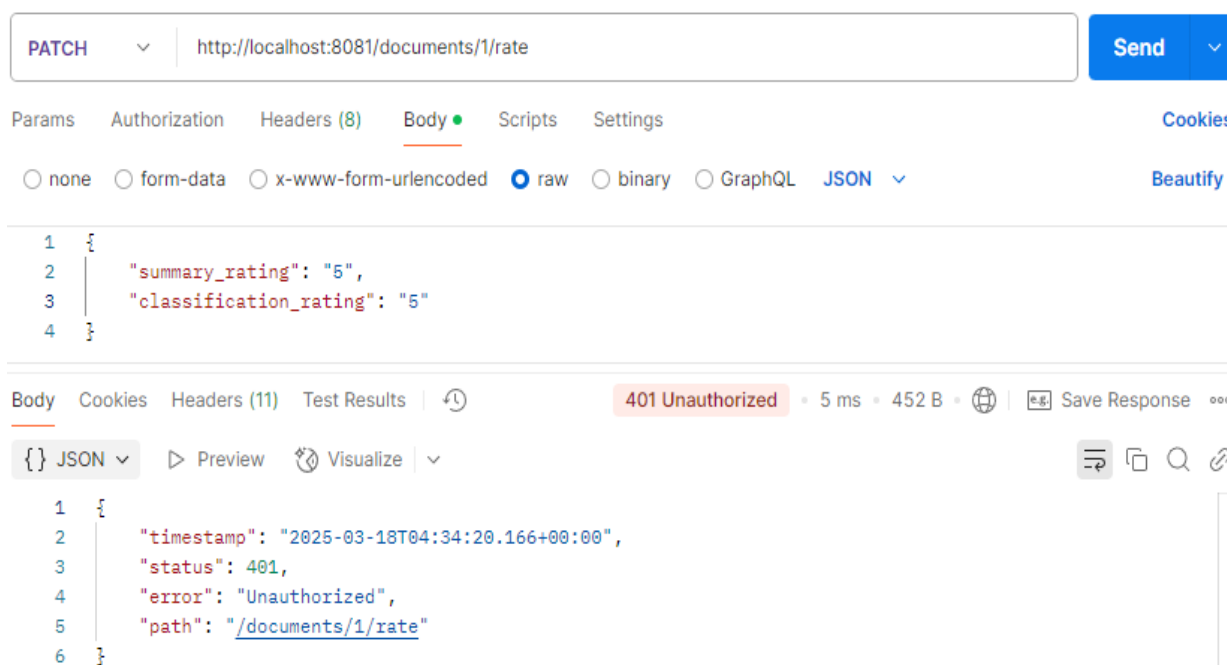


Рисунок 25 - Пример запроса оценки документа по id, ответ 401

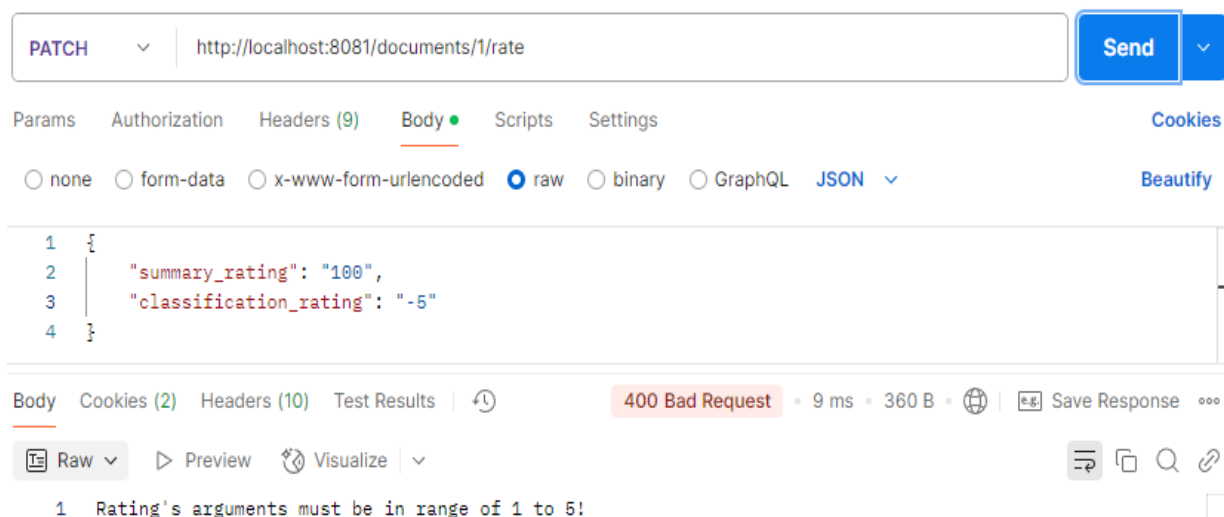


Рисунок 26 - Пример запроса оценки документа по id, ответ 400

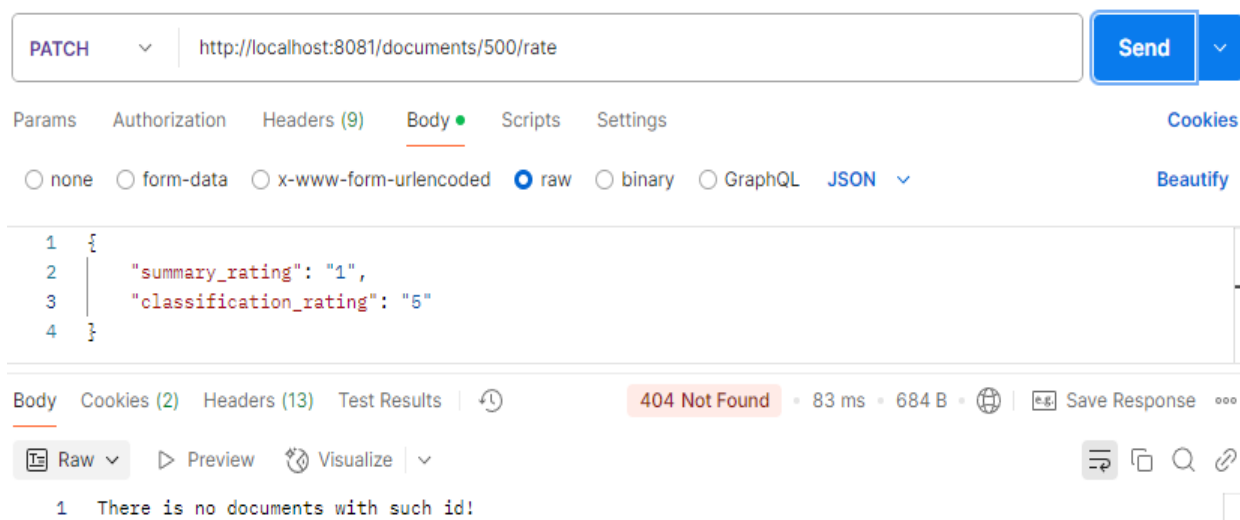


Рисунок 27 - Пример запроса оценки документа по id, ответ 404

### 3.2.7 Поиск документов

Данный функционал позволяет сначала отфильтровать документы на основе метаданных, где документ соответствующая метаданная должна входить в список соответствующей метаданной в запросе, если поле в запросе пустое или отсутствует, то проверка на данный вид метаданной не происходит и считается автоматически пройденной. После фильтрации выполняется один из трех видов поиска:

- Поиск по id – находит документ с интересующим идентификатором,
- Точный поиск – ищет документы по точному совпадению с поисковым запросом,
- Синтаксический – ищет документы по всему тексту на совпадение с поисковым запросом.

Также, для ускорения получения результатов поиска пользователем, функционал предусматривает пагинацию, выводя результаты поиска постепенно.

Чтобы произвести поиск, необходимо выполнить запрос – POST “/documents/search”, передав в тело запроса параметры поиска и фильтрации в соответствии со схемой поиска документа.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось оценить документ (смотри рисунок 28),
- 404 – попытка произвести поиск по id, которого нет в БД (смотри рисунок 29).

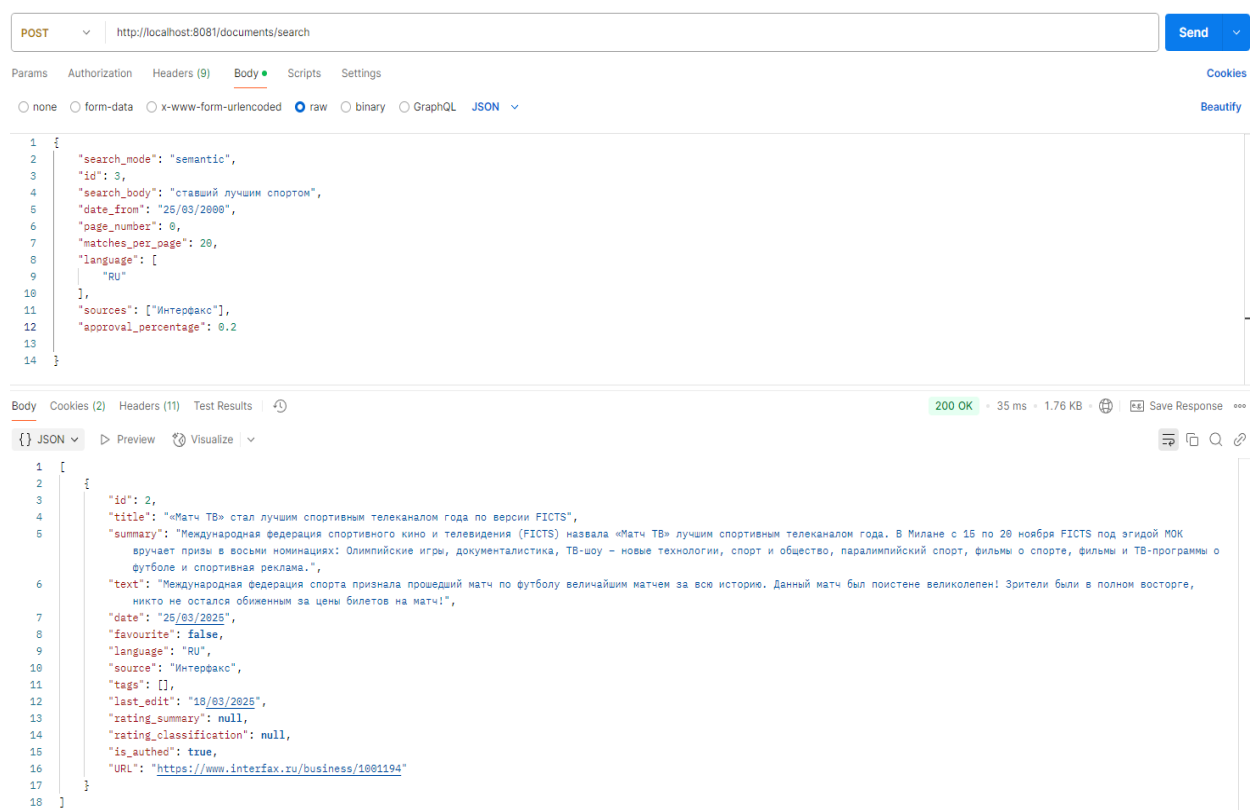


Рисунок 28 - Пример запроса фильтрации и поиска документа по id, ответ 200

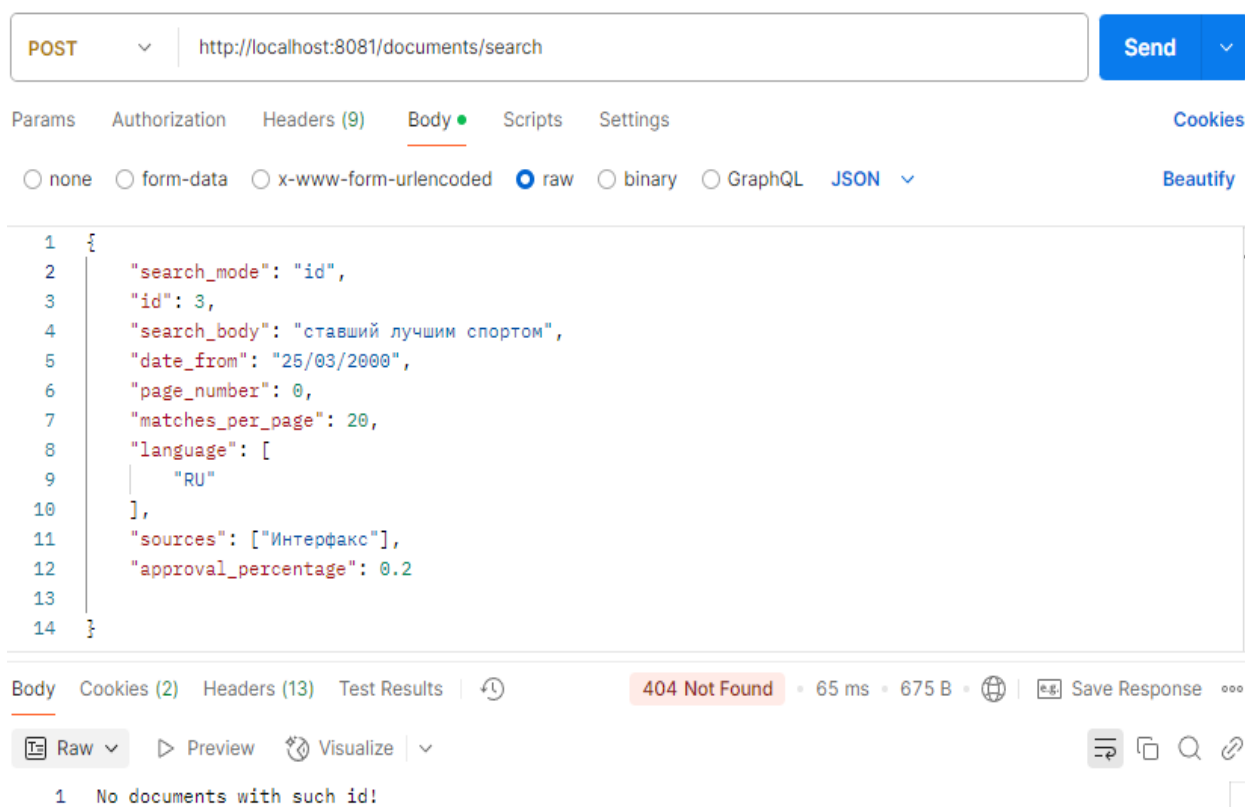


Рисунок 29 - Пример запроса фильтрации и поиска документа по id, ответ 404

После успешного получения документа, в тело ответа передается один объект, соответствующий схеме получения документа. Все особенности получения данных документа, связанных с пользователем, идентичны особенностям получения списка документов и документа по id.

### 3.2.8 Добавление документа в папку

Для структуризации новостных статей предоставляется функционал добавления документов в папки. При этом один и тот же документ в одной папке может присутствовать лишь один раз, поэтому при попытке повторного добавления вернется сообщение 200, но новый документ в папку не добавится. Чтобы добавить документ в папку, необходимо выполнить запрос – POST “  
/documents/{docId}/put\_into/{packageName}”, где docId – идентификатор документа, который необходимо переместить в папку, а packageName – имя папки, в которую необходимо переместить документ.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось добавить документ в папку (смотри рисунок 30),

- 401 – попытка добавить документ в папку, не авторизовавшись, что запрещено (смотри рисунок 31),
- 404 – попытка добавить документ с некорректным id или в папку с несуществующим названием (смотри рисунок 32 и рисунок 33).

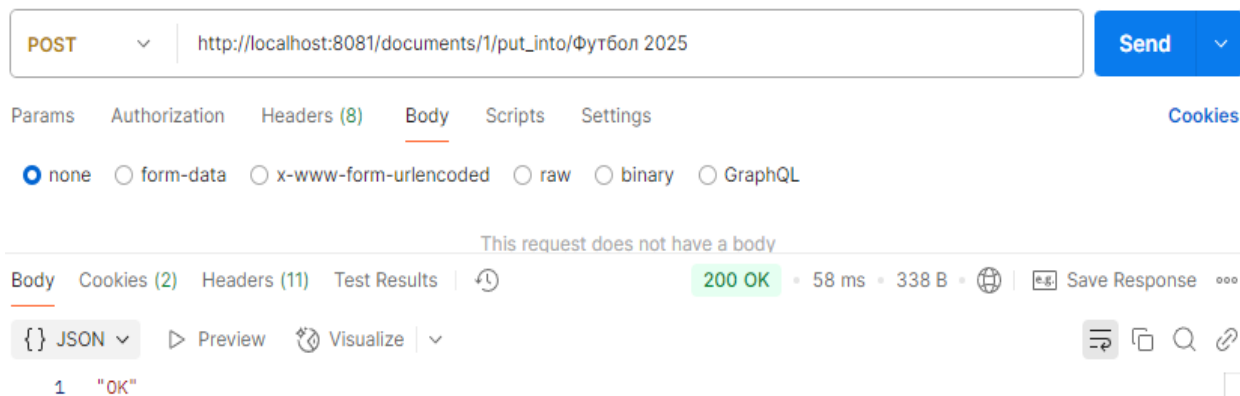


Рисунок 30 - Пример запроса документа в папку по id, ответ 200

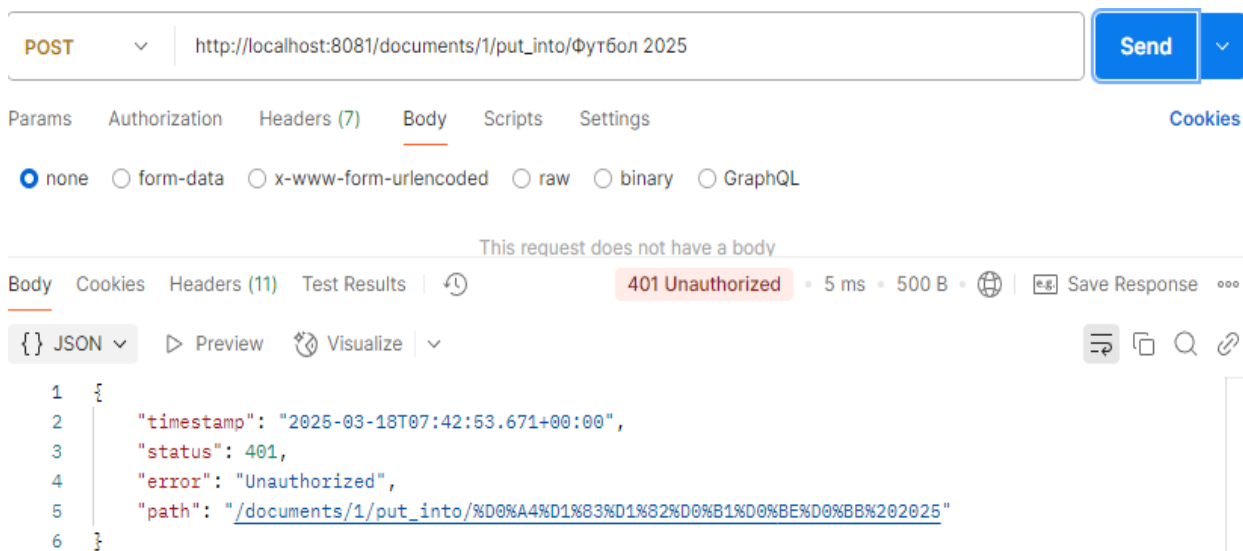


Рисунок 31 - Пример запроса документа в папку по id, ответ 401

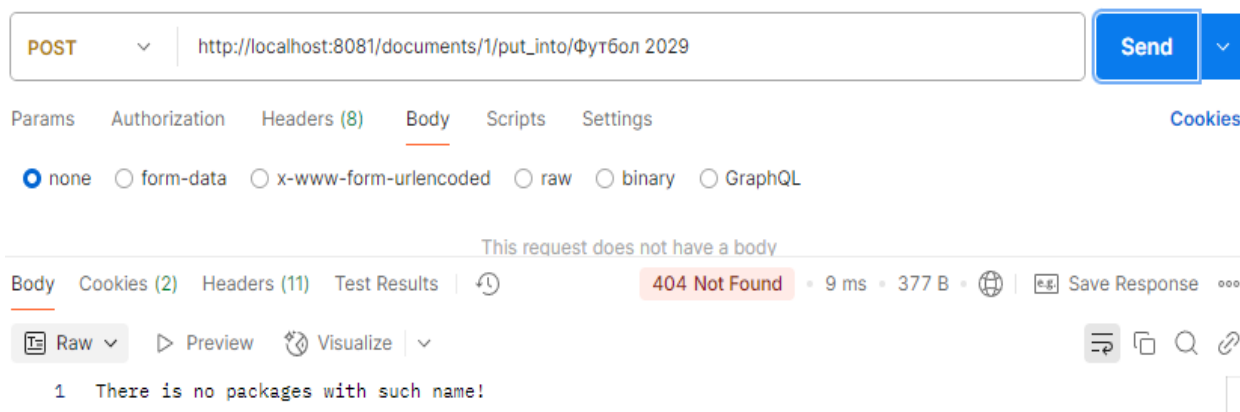


Рисунок 32 - Пример запроса документа в папку по id, ответ 404, некорректное имя папки

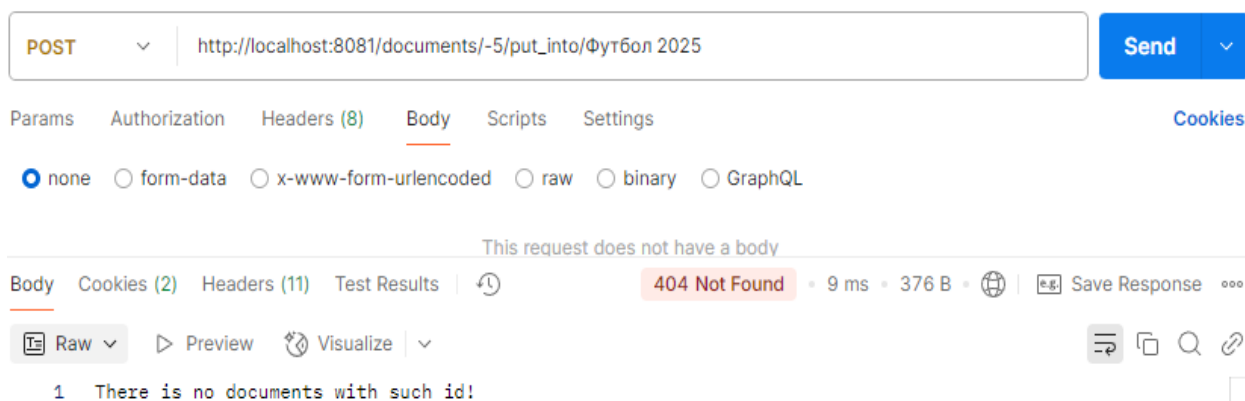


Рисунок 33 - Пример запроса документа в папку по id, ответ 404, некорректный id документа

### 3.2.9 Удаление документа из папки

Помимо добавления документа в папки, предоставляется возможность удаления документа из папки. При этом, также как и для добавления документа в папку, попытка удалить документ из папки, которого в ней нет вернет ответ 200, но никак не повлияет на записи в БД. Чтобы удалить документ из папки, необходимо воспользоваться запросом – POST `“/documents/{docId}/remove_from/{packageName}”`, где `docId` – идентификатор документа, который необходимо удалить из папки, а `packageName` – имя папки, из которой необходимо удалить документ.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить документ из папки (смотри рисунок 34),
- 401 – попытка удалить документ из папки, не авторизовавшись, что запрещено (смотри рисунок 35),
- 404 – попытка удалить документ с некорректным id или из папки с несуществующим названием (смотри рисунок 36).

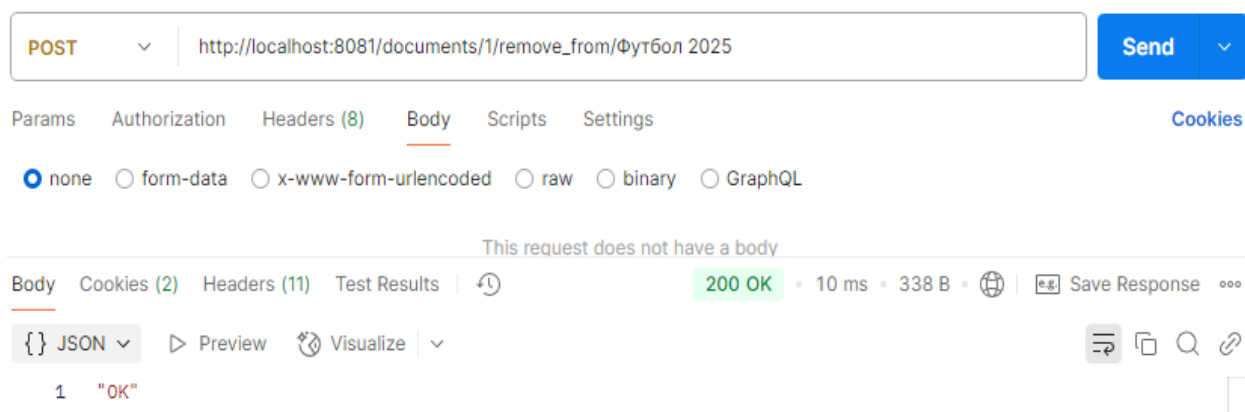


Рисунок 34 - Пример запроса удаления документа из папки по id, ответ 200

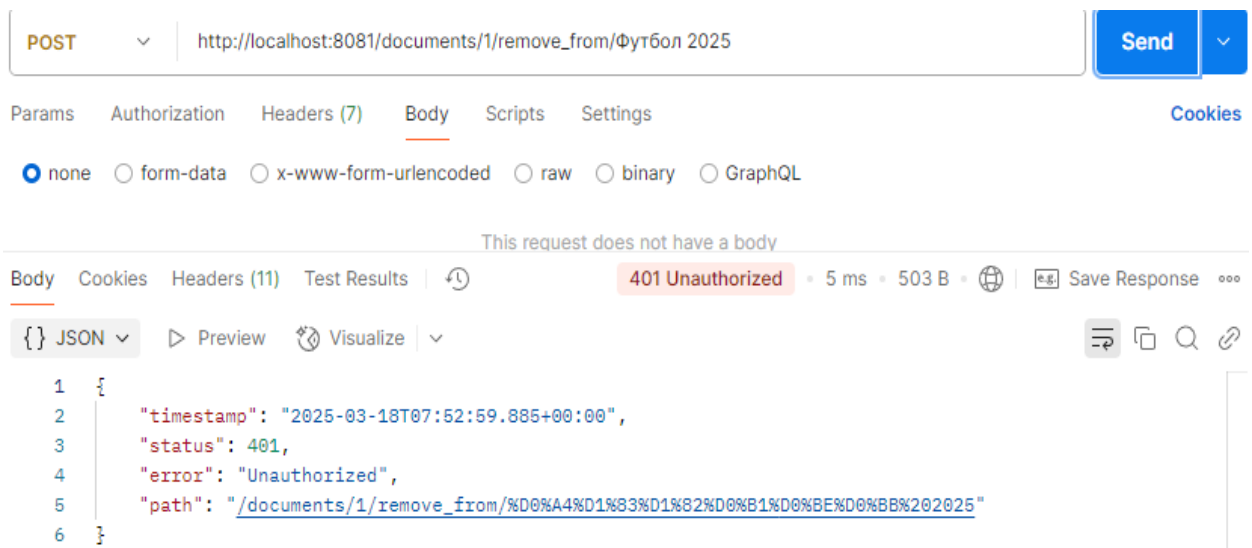


Рисунок 35 - Пример запроса удаления документа из папки по id, ответ 401

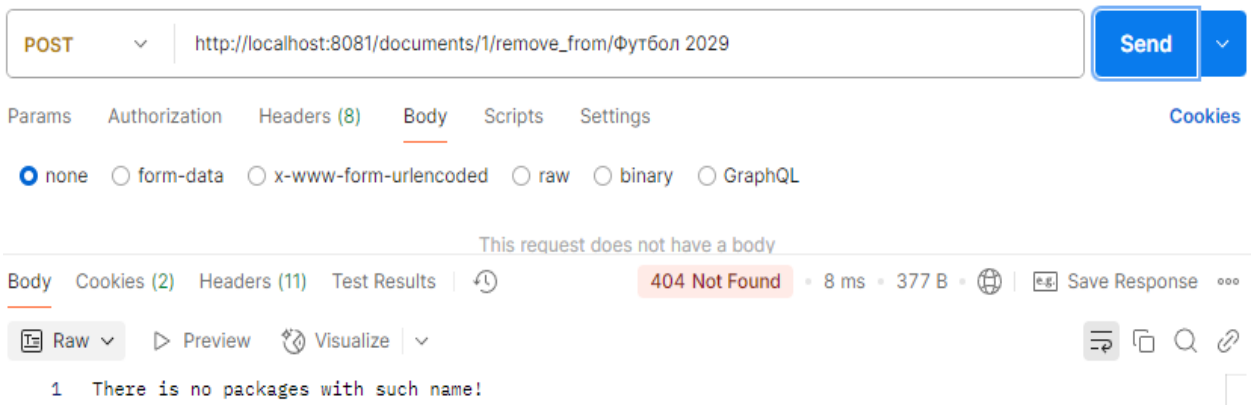


Рисунок 36 - Пример запроса удаления документа из папки по id, ответ 404, некорректное имя папки

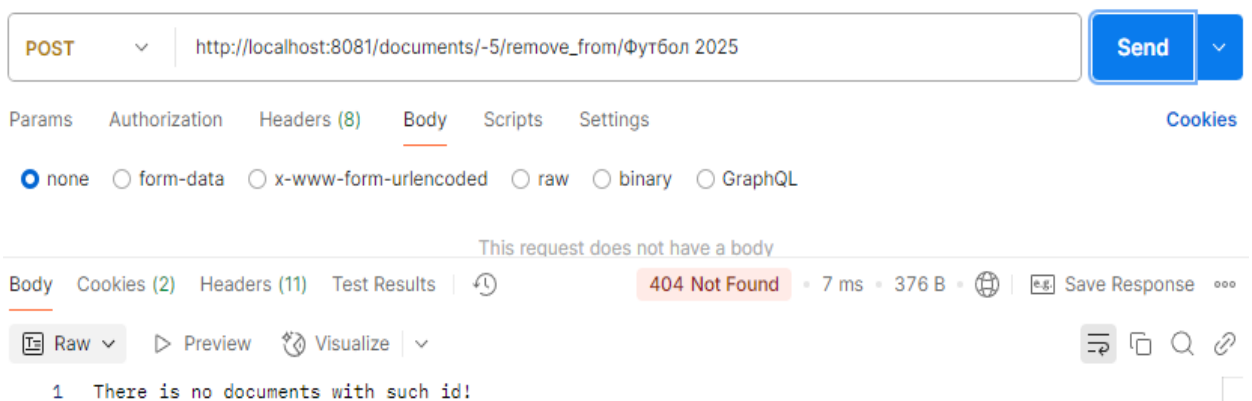


Рисунок 37 - Пример запроса удаления документа из папки по id, ответ 404, некорректный id документа

### 3.2.10 Добавление документа в избранное

Поскольку на мобильном приложении нет функционала управления жизненным циклом папок, но есть папки “Избранное”, по одной на каждого пользователя, то

реализован упрощенный способ добавления документов в папку “Избранное”, все особенности сохраняются, как при обычном добавлении документа в папку. Чтобы добавить статью в “Избранное”, необходимо воспользоваться запросом – POST “/documents/{id}/add\_to\_favourites”, где docId – идентификатор документа, который необходимо переместить в папку “Избранное”.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось добавить документ в папку “Избранное” (смотри рисунок 38),
- 401 – попытка добавить документ в папку “Избранное”, не авторизовавшись, что запрещено (смотри рисунок 39),
- 404 – попытка добавить документ с некорректным id в папку “Избранное” (смотри рисунок 40).

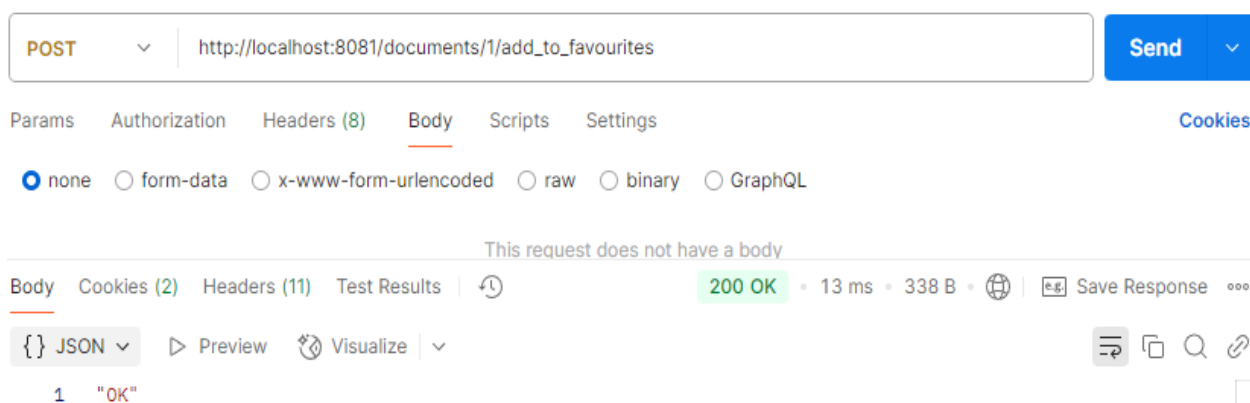


Рисунок 38 - Пример запроса добавление документа в избранное по id, ответ 200

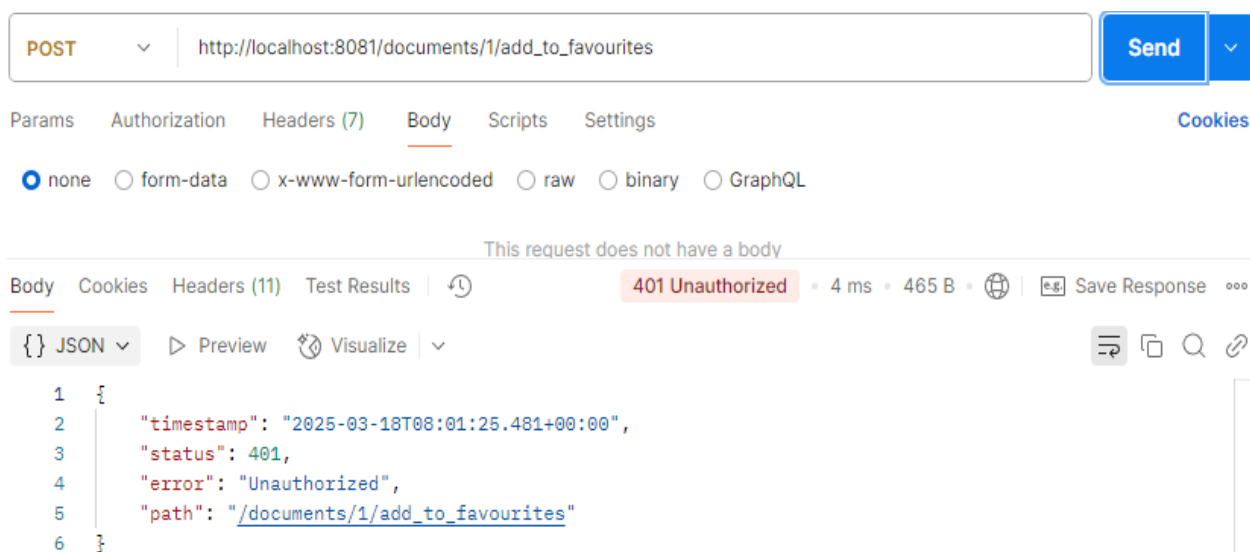


Рисунок 39 - Пример запроса добавление документа в избранное по id, ответ 401

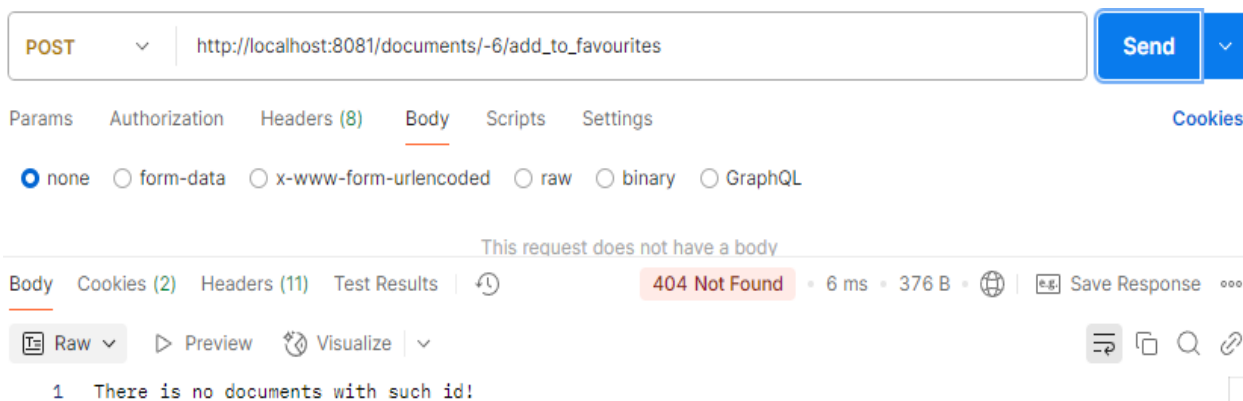


Рисунок 40 - Пример запроса добавление документа в избранное по id, ответ 404

### 3.2.11 Удаление документа из избранного

Также для упрощенного доступа мобильного приложения, добавлен функционал удаления документа из папки “Избранное”. Особенности удаления из папки “Избранное”, соответствуют особенностям удаления из любой другой папки. Чтобы удалить документ из папки “Избранное”, необходимо воспользоваться запросом – POST `“/documents/{id}/remove_from_favourites”`, где docId – идентификатор документа, который необходимо удалить из папки “Избранное”.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить документ из папки “Избранное” (смотри рисунок 41),
- 401 – попытка удалить документ из папки “Избранное”, не авторизовавшись, что запрещено (смотри рисунок 42),
- 404 – попытка удалить документ с некорректным id из папки “Избранное” (смотри рисунок 43).

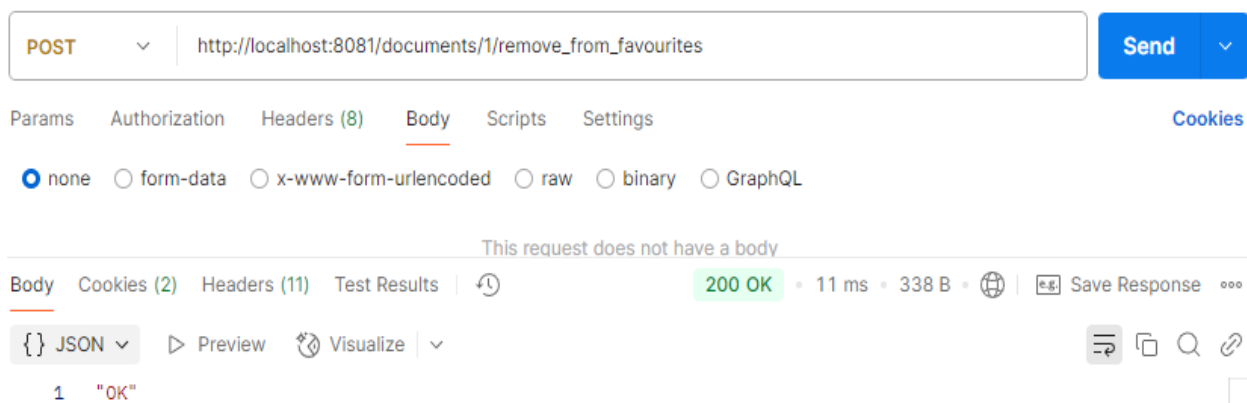


Рисунок 41 - Пример запроса удаления документа из избранного по id, ответ 200

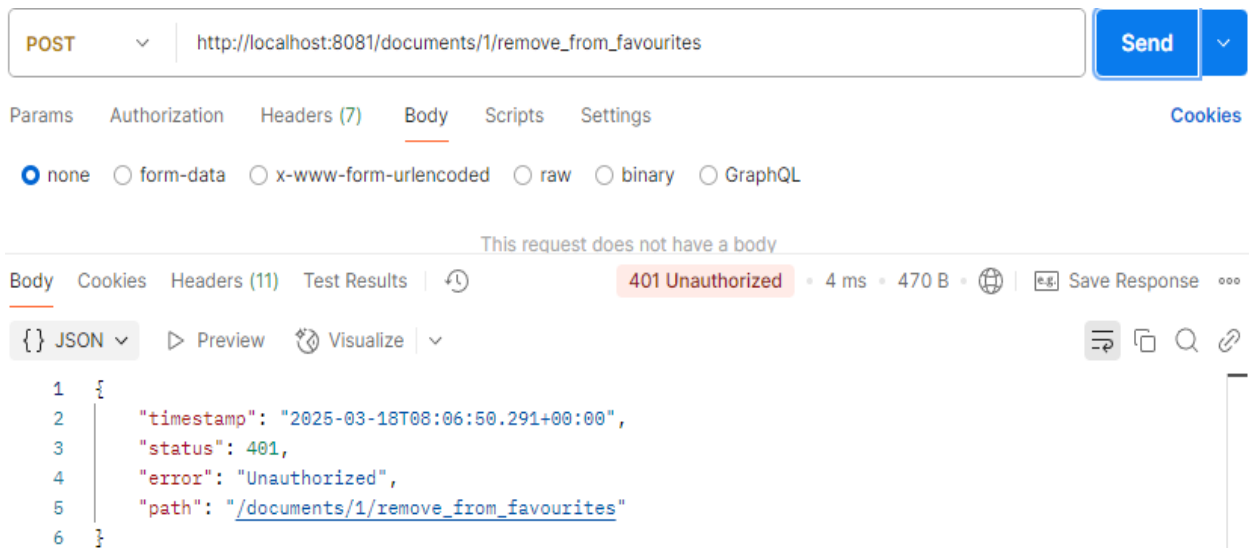


Рисунок 42 - Пример запроса удаления документа из избранного по id, ответ 401

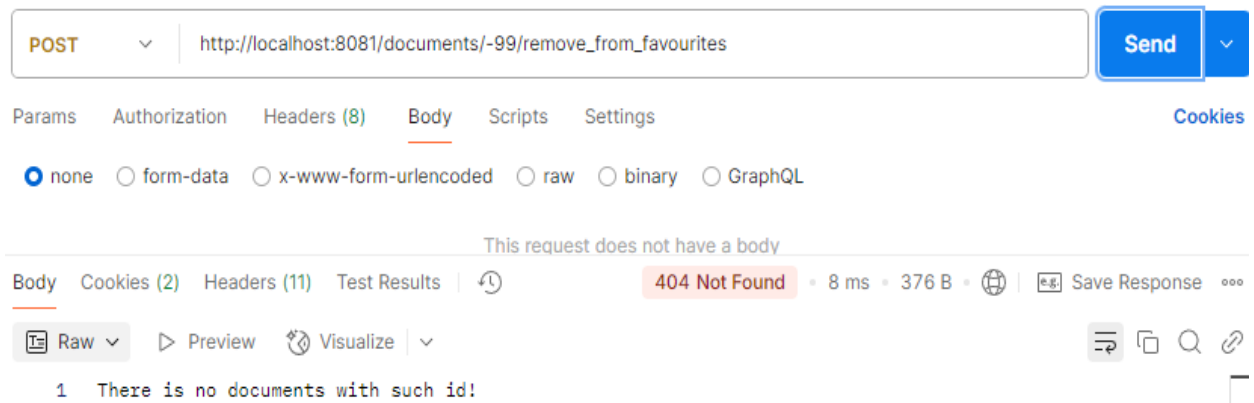


Рисунок 43 - Пример запроса удаления документа из избранного по id, ответ 404

### 3.2.12 Создание аннотации в документе

Стоит напомнить, что при удалении документа, все связанные с ним аннотации удаляются. Для создания аннотации в документе необходимо воспользоваться запросом - POST “/documents/{id}/annotate”, где id – идентификатор документа, который необходимо проаннотировать, также данные о создаваемой аннотации необходимо передать в теле запроса, в соответствии со схемой создания аннотации, данные о пользователе берутся из access токена.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса, в случае успеха в тело ответа передается целочисленный идентификатор созданной аннотации:

- 200 – успешно получилось создать аннотацию к документу (смотри рисунок 44),
- 401 – попытка аннотирования документа, не авторизовавшись, что запрещено (смотри рисунок 45),

- 404 – попытка аннотировать документ с некорректным id (смотри рисунок 46).

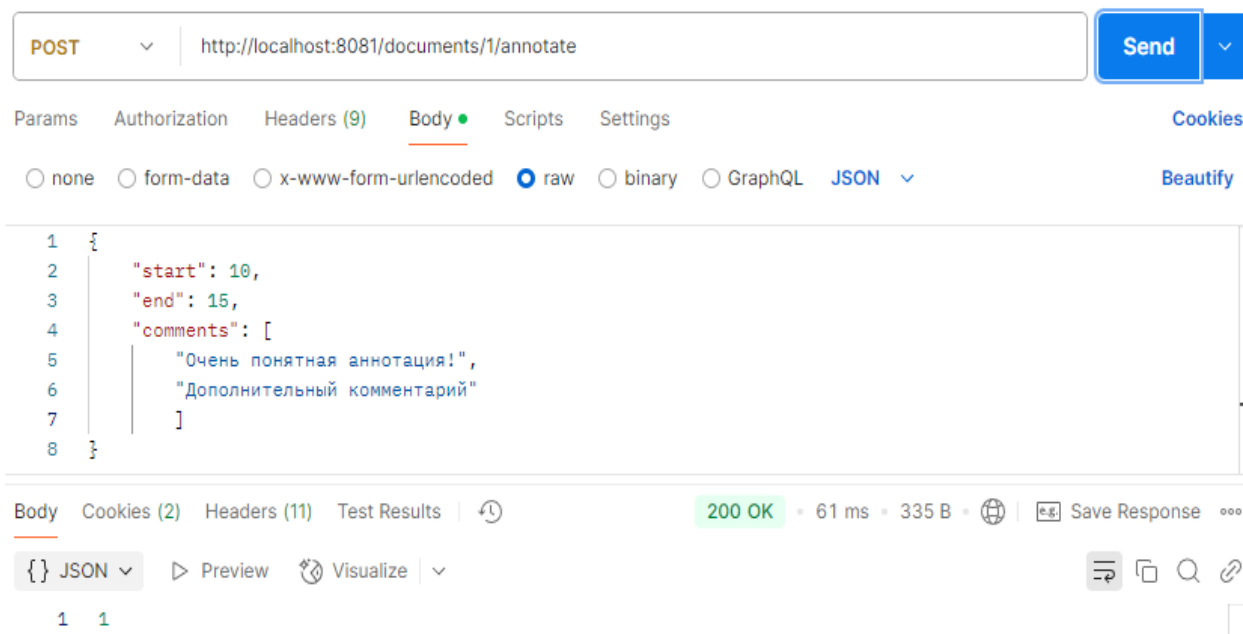


Рисунок 44 - Пример запроса аннотирования документа по id, ответ 200

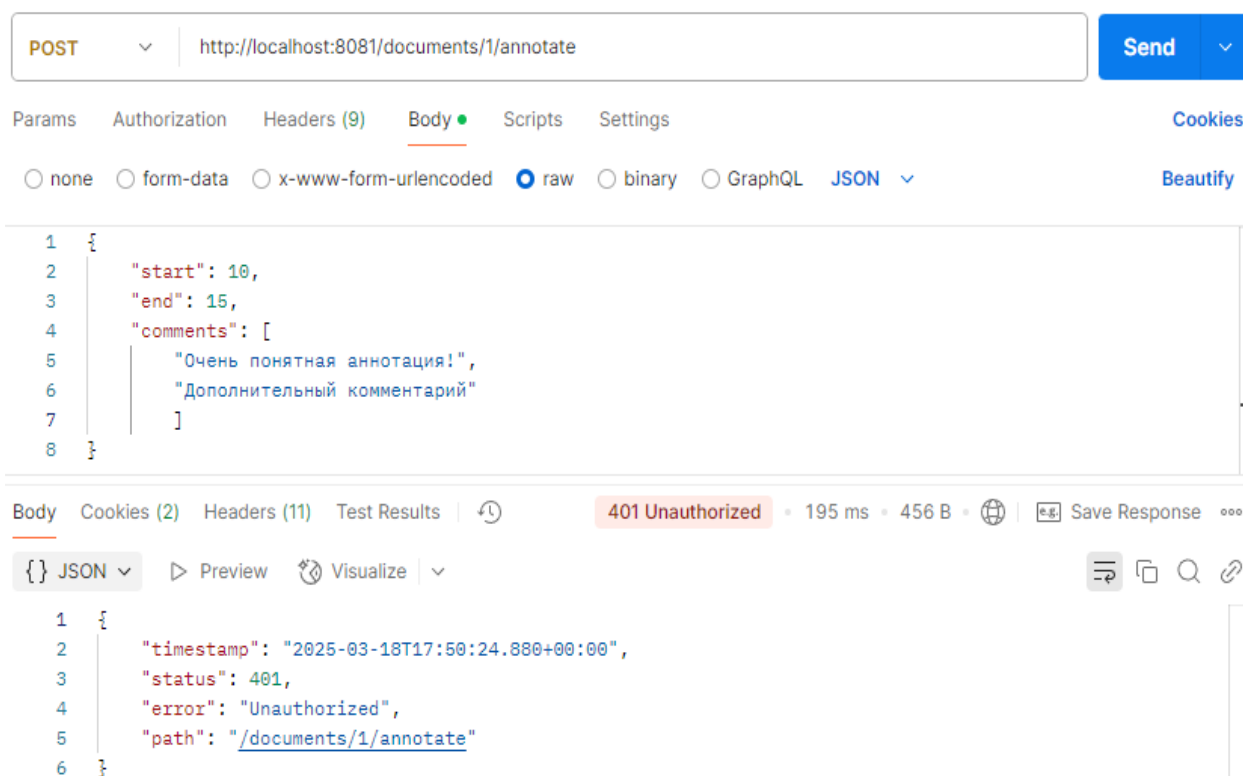


Рисунок 45 - Пример запроса аннотирования документа по id, ответ 401

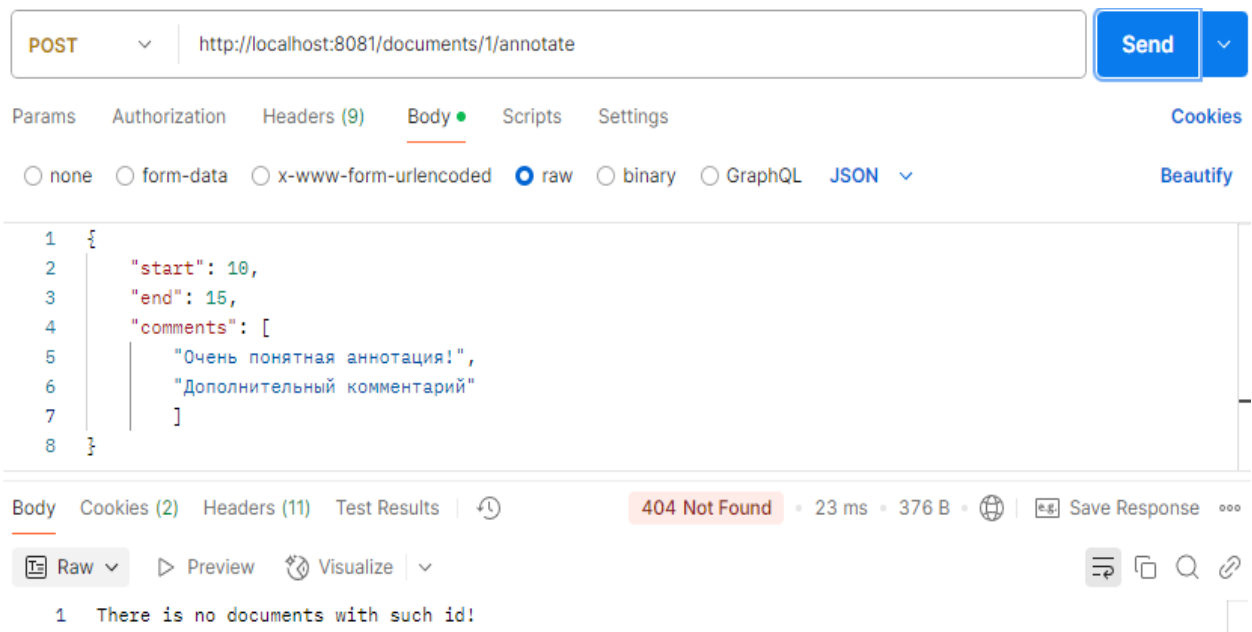


Рисунок 46 - Пример запроса аннотирования документа по id, ответ 404

### 3.2.13 Удаление аннотации из документа

Чтобы удалить аннотацию с документа, необходимо воспользоваться запросом – DELETE “/documents/{docId}/annotate/{annotationId}”, где docId – идентификатор документа, у которого удаляют аннотацию, а annotationId – идентификатор аннотации, которую удаляют, данные о пользователе получаются из access токена.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить аннотации к документу (смотри рисунок 47),
- 401 – попытка удаления аннотации к документу, не авторизовавшись, что запрещено (смотри рисунок 48),
- 404 – попытка удалить аннотацию к документу с некорректным id документа или некорректным id аннотации (смотри рисунок 49),
- 400 – попытка удалить аннотацию, которая не принадлежит пользователю или документу (смотри рисунок 50).

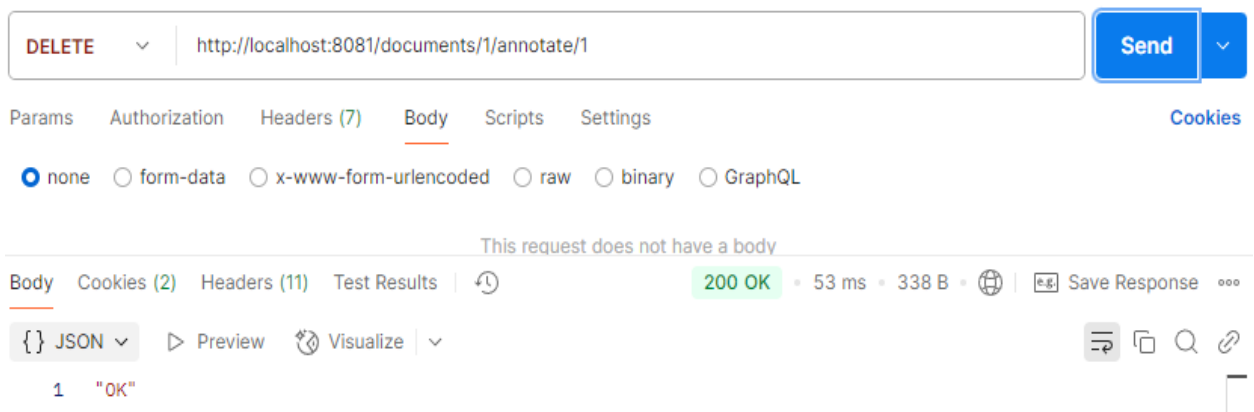


Рисунок 47 - Пример запроса удаления аннотирования документа по id, ответ 200

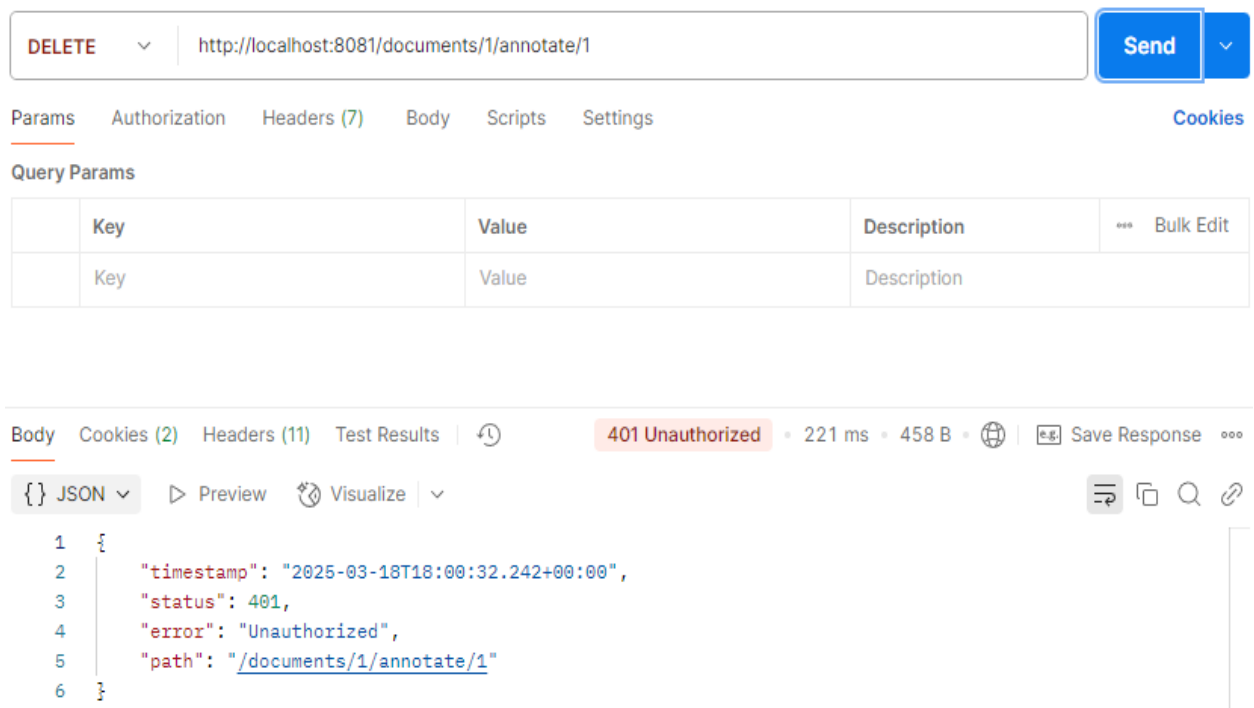


Рисунок 48 - Пример запроса удаления аннотирования документа по id, ответ 401

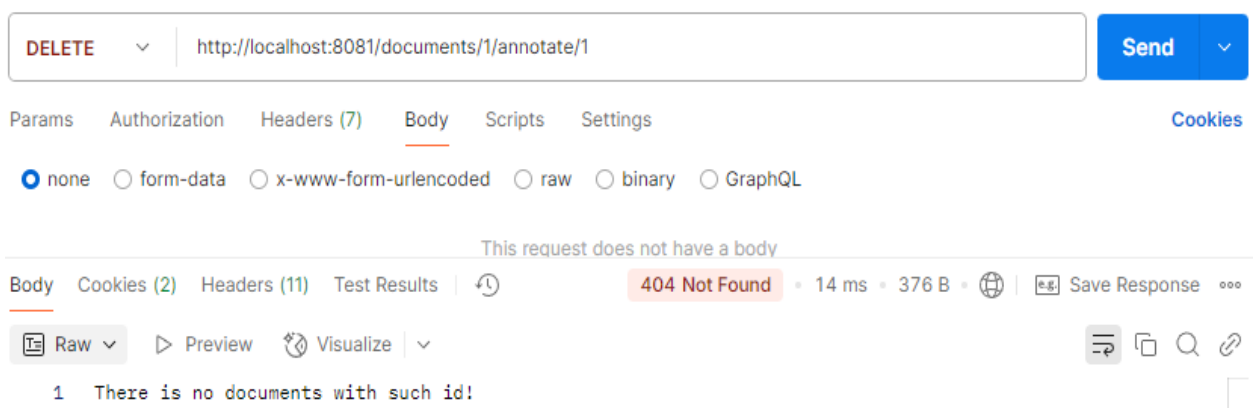


Рисунок 49 - Пример запроса удаления аннотирования документа по id, ответ 404

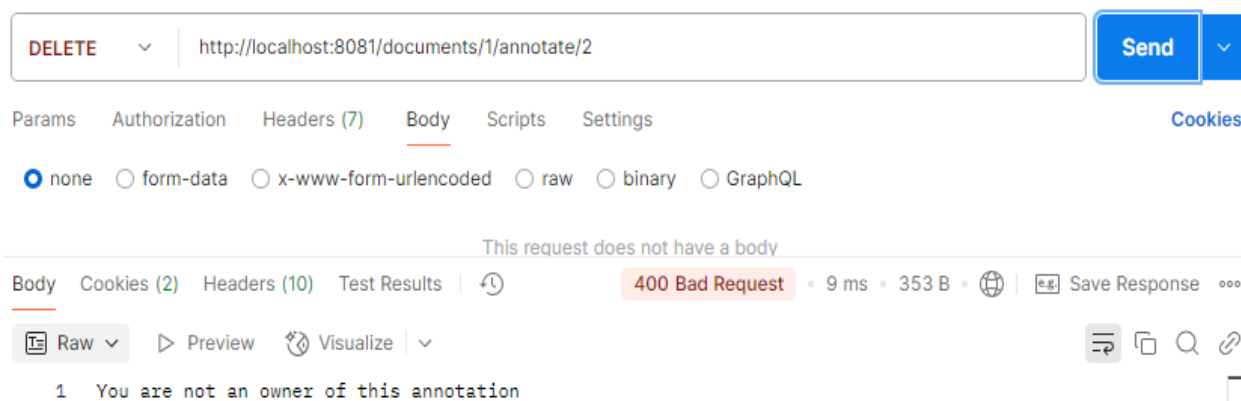


Рисунок 50 - Пример запроса удаления аннотирования документа по id, ответ 400

### 3.2.14 Создание комментария к документу

Стоит напомнить, что при удалении документа, все связанные с ним комментарии удаляются. Для создания комментария в документе необходимо воспользоваться запросом - POST “/documents/{id}/comment”, где id – идентификатор документа, который необходимо прокомментировать, также данные о создаваемом комментарии необходимо передать в теле запроса, в соответствии со схемой создания комментария, данные о пользователе берутся из access токена.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось создать комментарий к документу (смотри рисунок 51),
- 401 – попытка комментирования документа, не авторизовавшись, что запрещено (смотри рисунок 52),
- 404 – попытка комментировать документ с некорректным id (смотри рисунок 53).

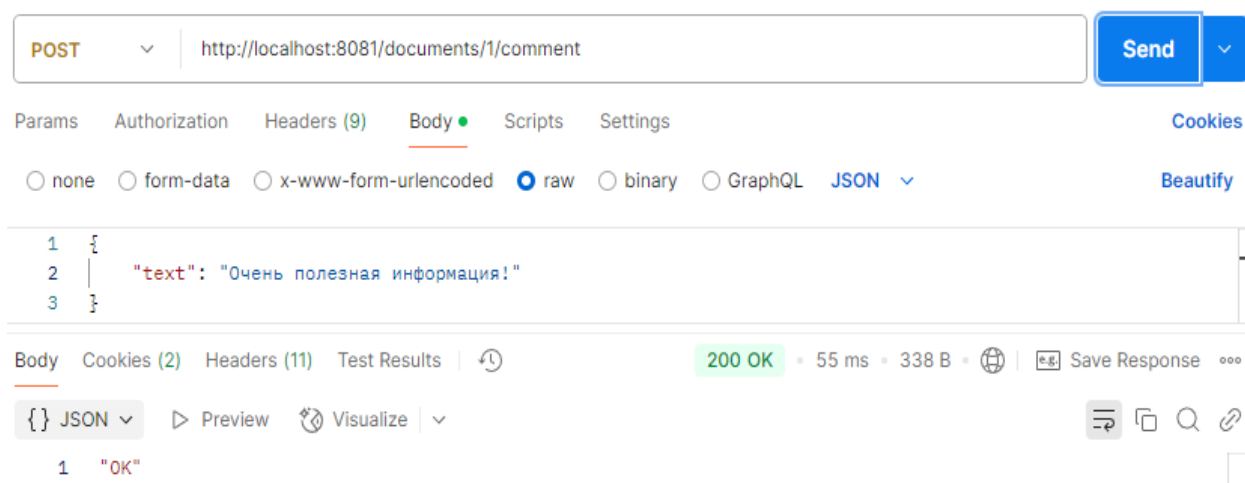


Рисунок 51 - Пример запроса комментирования документа по id, ответ 200

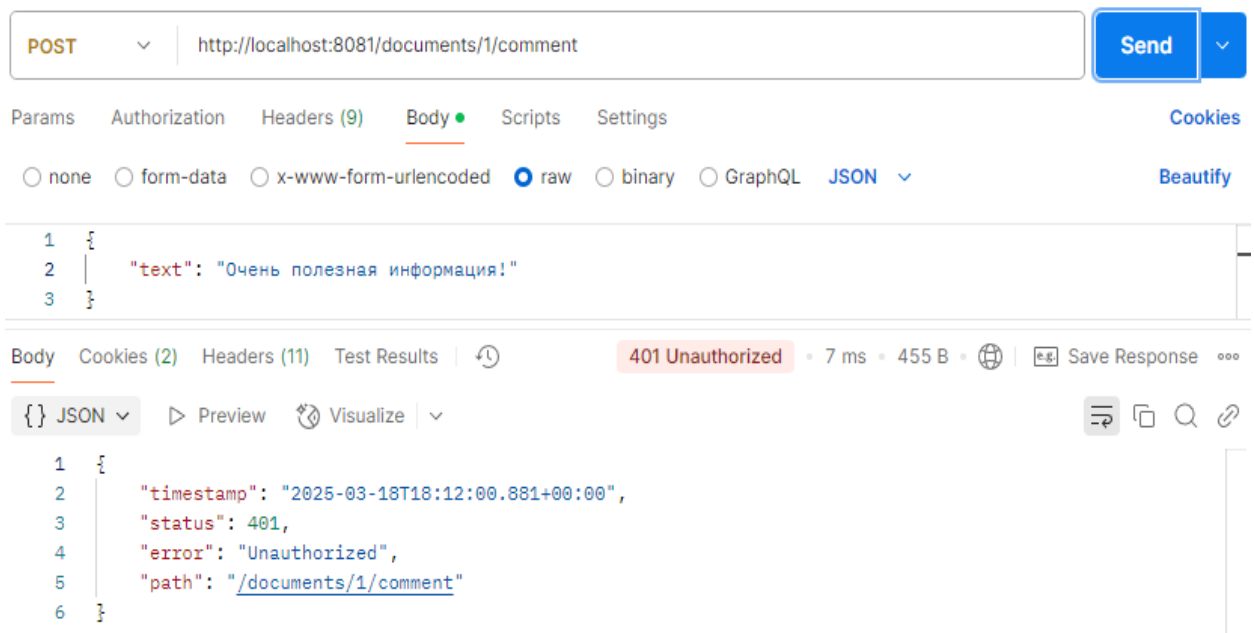


Рисунок 52 - Пример запроса комментирования документа по id, ответ 401

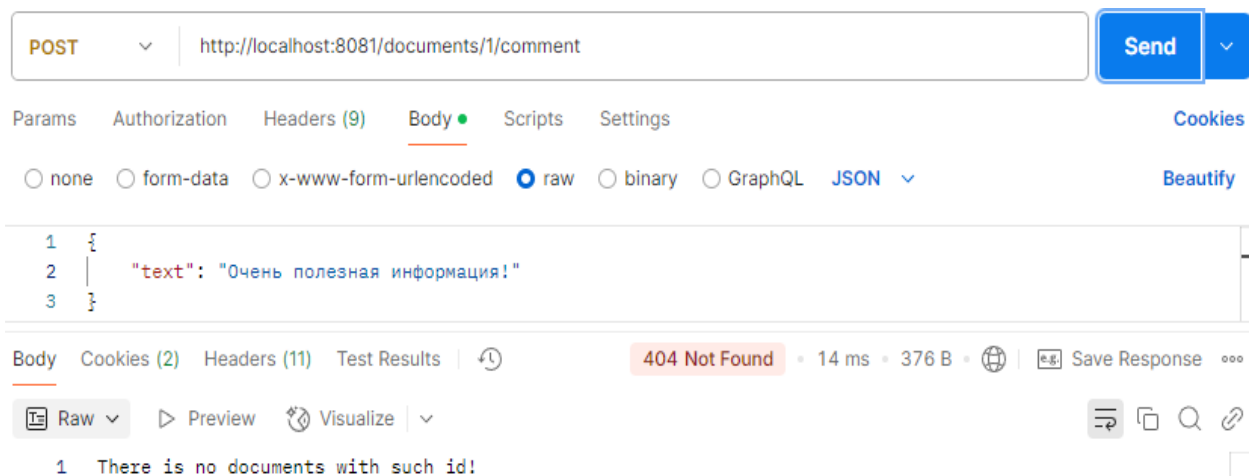


Рисунок 53 - Пример запроса комментирования документа по id, ответ 404

### 3.2.15 Удаление комментария к документу

Чтобы удалить комментарий с документа, необходимо воспользоваться запросом – DELETE “`/documents/{docId}/annotate/{commentId}`”, где `docId` – идентификатор документа, у которого удаляют комментарий, а `commentId` – идентификатор комментария, который удаляют, данные о пользователе получаются из access токена.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить комментарий к документу (смотри рисунок 54),
- 401 – попытка удаления комментария к документу, не авторизовавшись, что запрещено (смотри рисунок 55),

- 404 – попытка удалить комментарий к документу с некорректным id документа или некорректным id комментария (смотри рисунок 56),
- 400 – попытка удалить комментарий, который не принадлежит пользователю или документу (смотри рисунок 57).

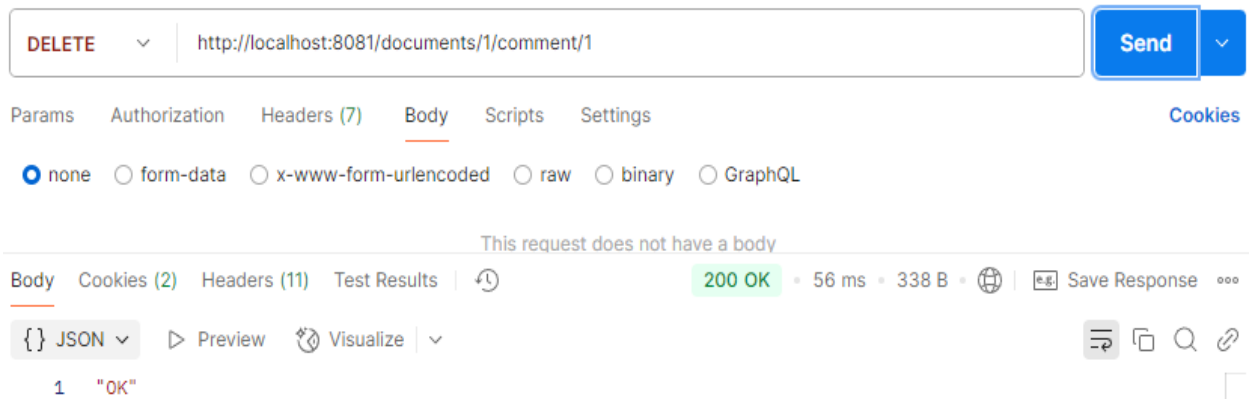


Рисунок 54 - Пример запроса удаления комментария документа по id, ответ 200

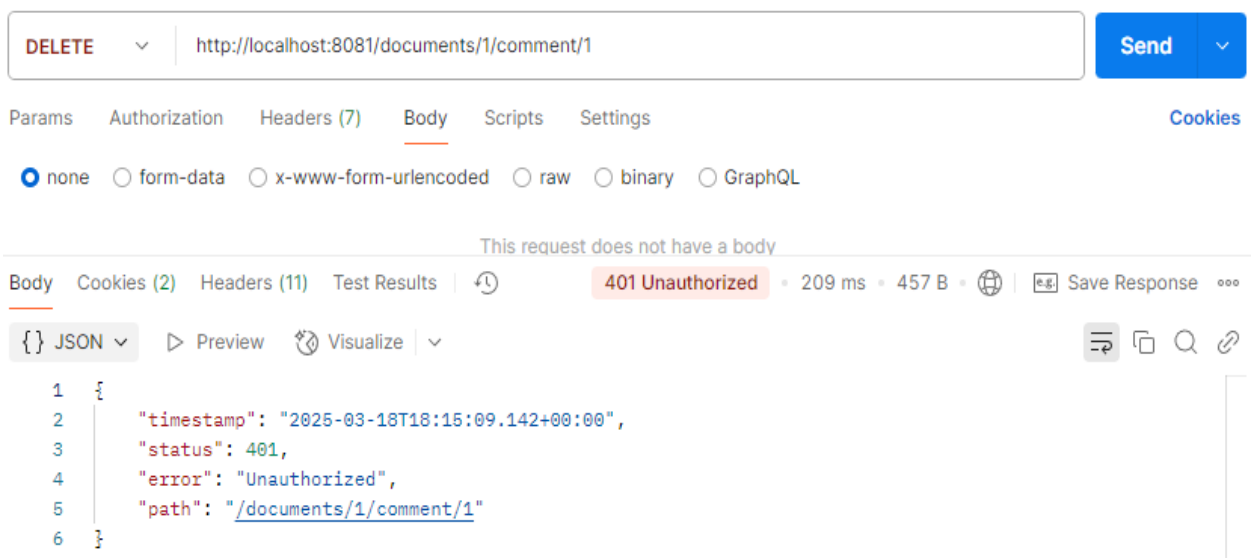


Рисунок 55 - Пример запроса удаления комментария документа по id, ответ 401

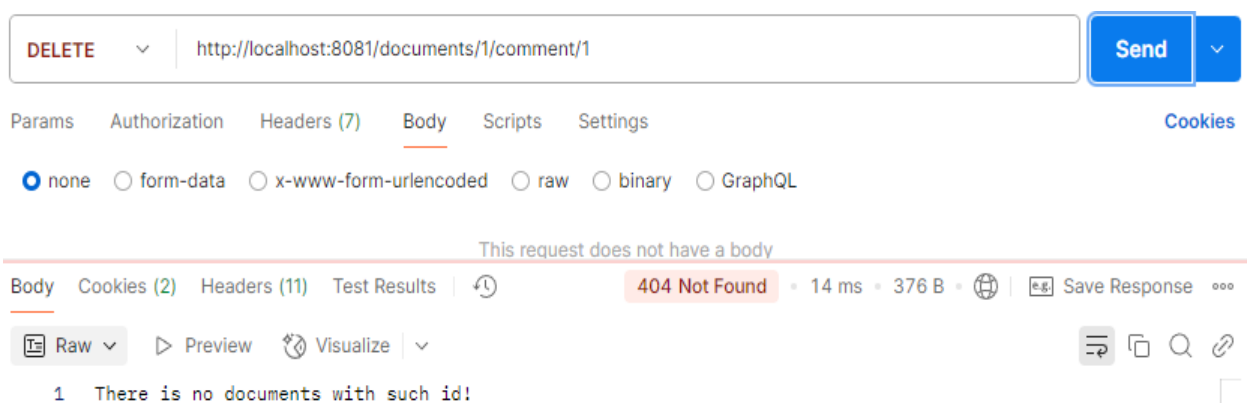


Рисунок 56 - Пример запроса удаления комментария документа по id, ответ 404

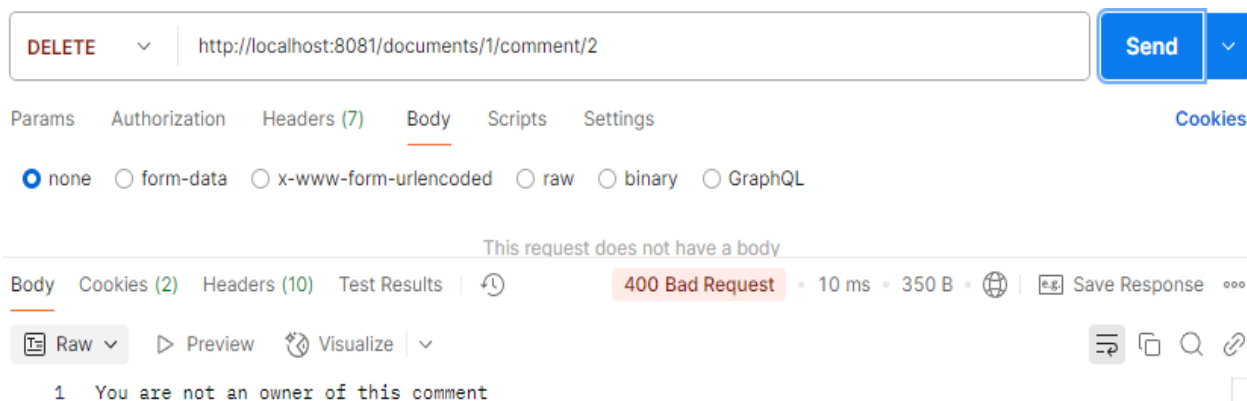


Рисунок 57 - Пример запроса удаления комментария документа по id, ответ 400

### 3.3 Функционал, связанный с управлением жизненным циклом тэгов

#### 3.3.1 Создание тэга

Для создания нового тега необходимо воспользоваться запросом – POST “/tags/create”, а в теле запроса передать название тега через поле “name”.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось создать новый тэг (смотри рисунок 58),
- 400 – тэг с таким именем уже существует (смотри рисунок 59).

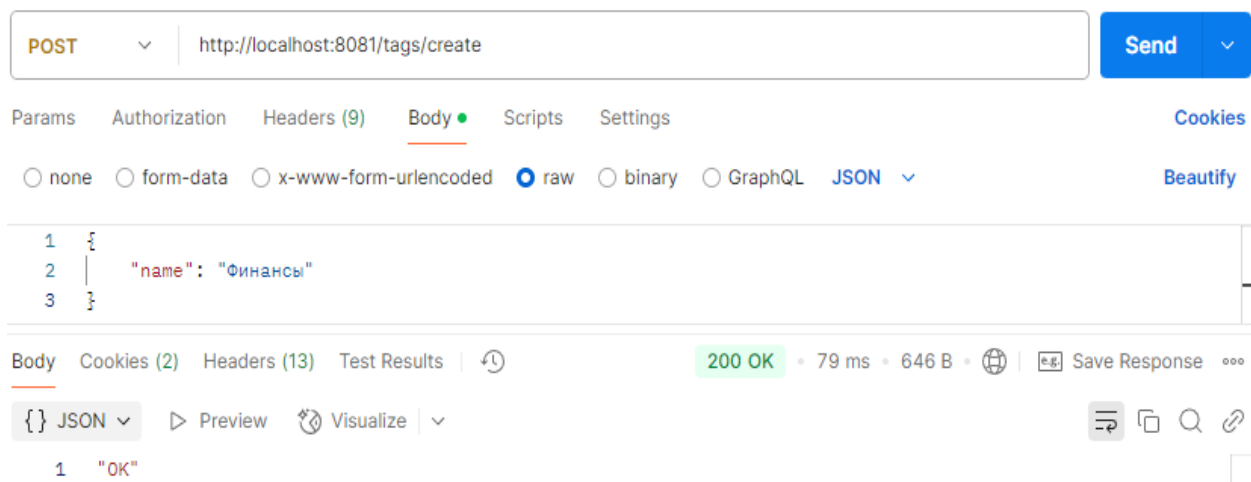


Рисунок 58 - Пример запроса создания тэга классификации, ответ 200

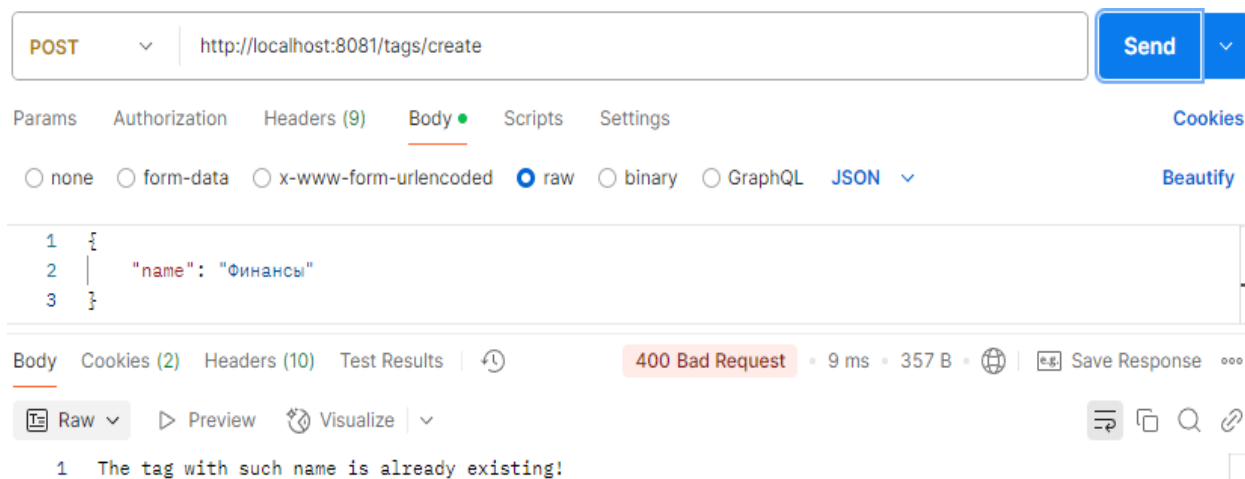


Рисунок 59 - Пример запроса создания тэга классификации, ответ 400

### 3.3.2 Удаление тэга по id

Для удаления тега по id необходимо воспользоваться запросом – DELETE “/tags/{id}”, где id – идентификатор удаляемого тега. Стоит отметить, что при удалении тега, связанные документы не изменятся, но из их списков тегов пропадет удаленный.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить тэг по id (смотри рисунок 60),
- 404 – тэга с таким id не существует (смотри рисунок 61).

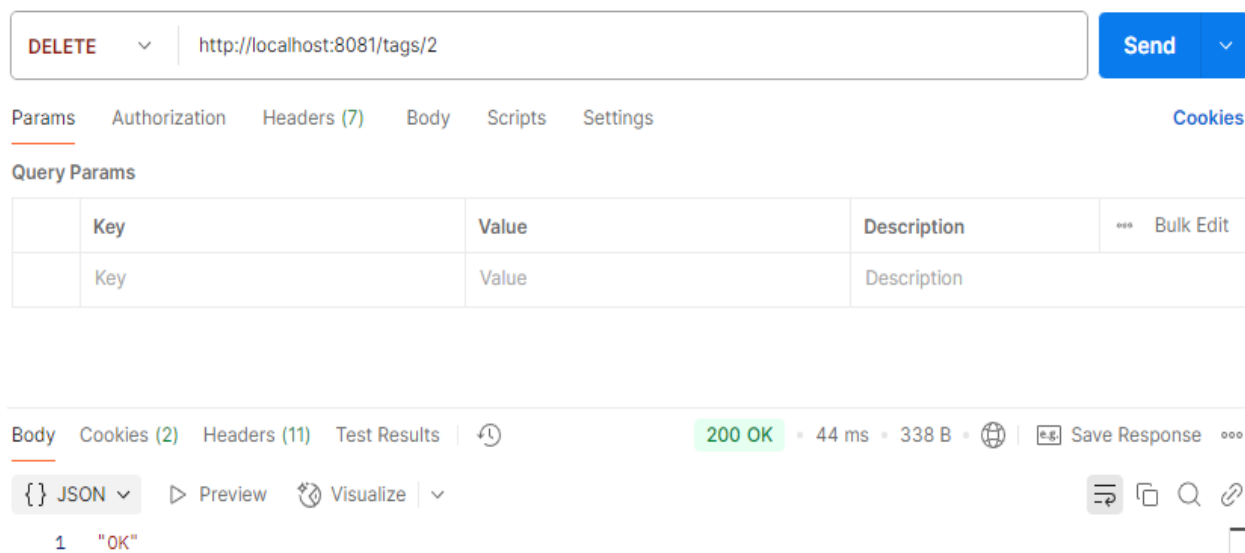


Рисунок 60 - Пример запроса удаления тэга классификации, ответ 200

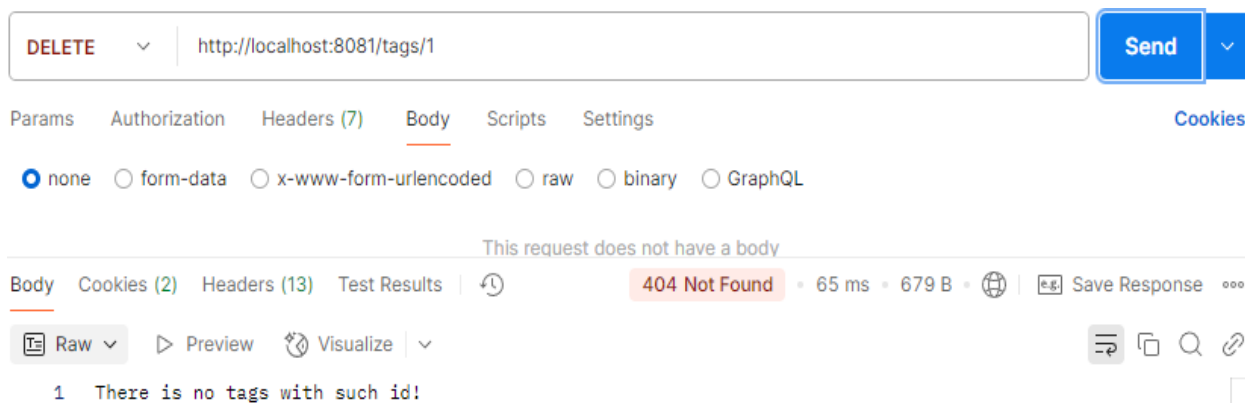


Рисунок 61 - Пример запроса удаления тэга классификации, ответ 404

### 3.4 Функционал, связанный с управлением жизненным циклом папок

#### 3.4.1 Создание папки

Названия папки уникальны для каждого пользователя, поэтому у одного пользователя не может быть две папки с одинаковым названием.

Для создания папки необходимо обратиться по запросу POST “/packages/create”, где в тело запроса передается значение имени новой папки, которое необходимо передать в поле “name”.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось создать новую папку (смотри рисунок 62),
- 401 – попытка создать папку, не авторизовавшись, что запрещено (смотри рисунок 63),
- 400 – попытка создать папку с существующим данным названием (смотри рисунок 64).

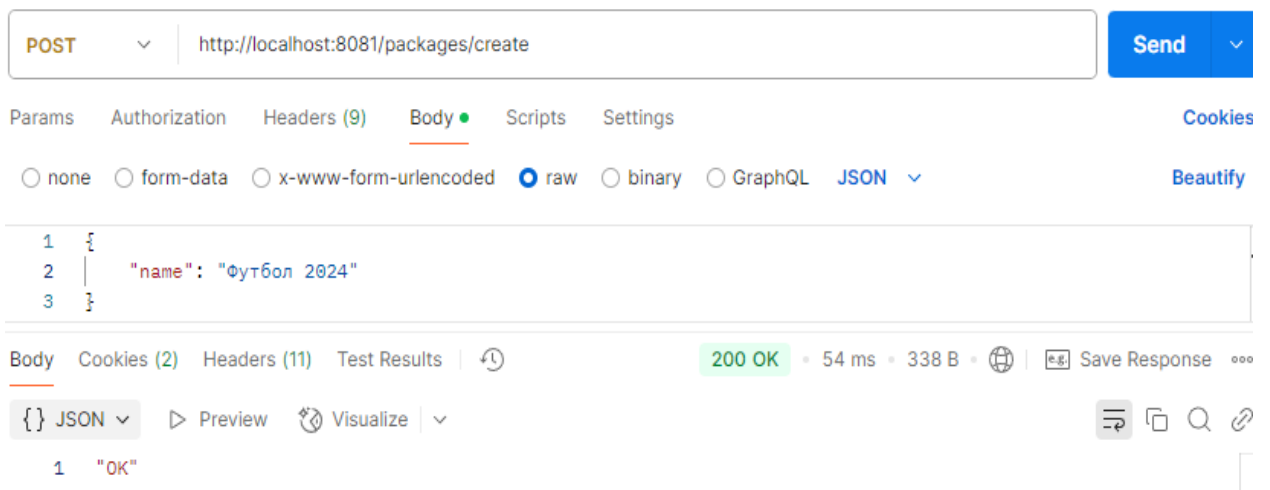


Рисунок 62 - Пример запроса создания папки, ответ 200

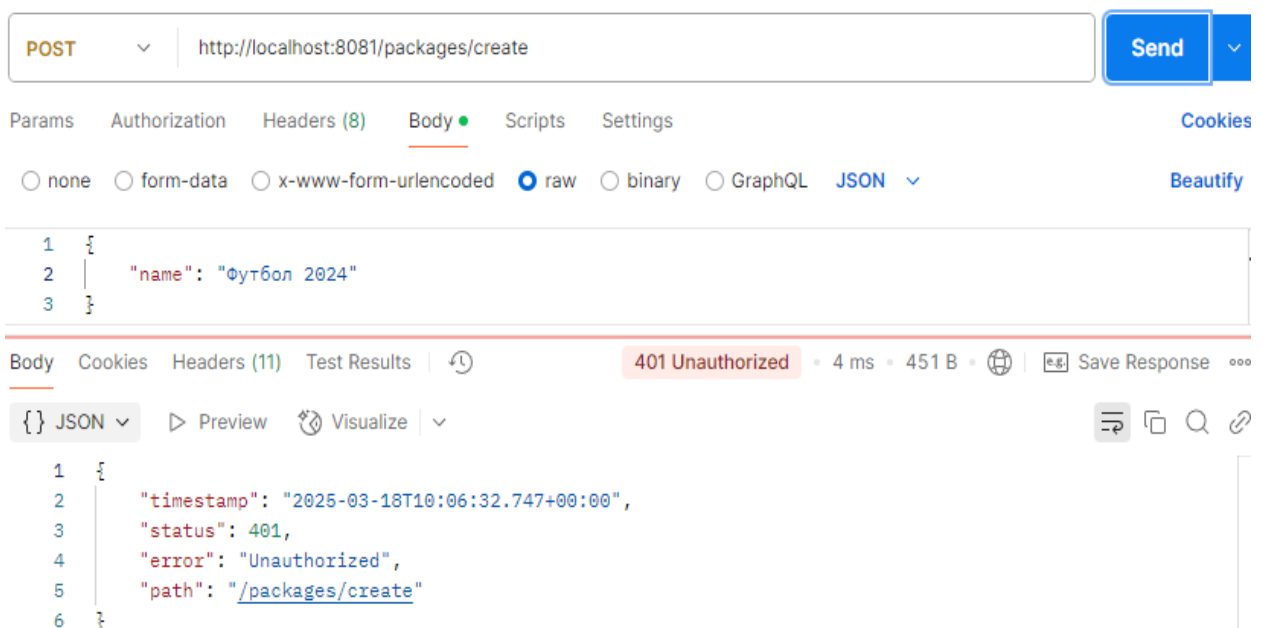


Рисунок 63 - Пример запроса создания папки, ответ 401

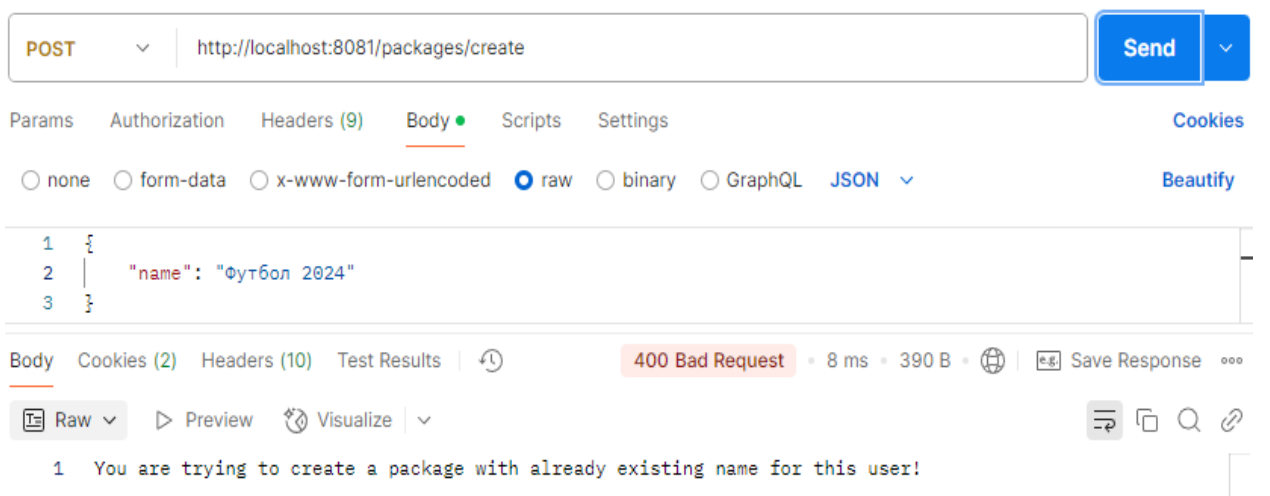


Рисунок 64 - Пример запроса создания папки, ответ 400

### 3.4.2 Удаление папки по названию

Помимо создания папки, присутствует функционал удаления папки по названию.

Чтобы удалить папку, необходимо воспользоваться запросом DELETE /packages/{name}, где name – имя удаляемой папки.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно получилось удалить папку по имени (смотри рисунок 65),
- 401 – попытка удалить папку, не авторизовавшись, что запрещено (смотри рисунок 66),
- 400 – попытка удалить неудаляемую папку, например “Избранное” (смотри рисунок 67),
- 404 – попытка удалить папку с несуществующим данным названием (смотри рисунок 68).

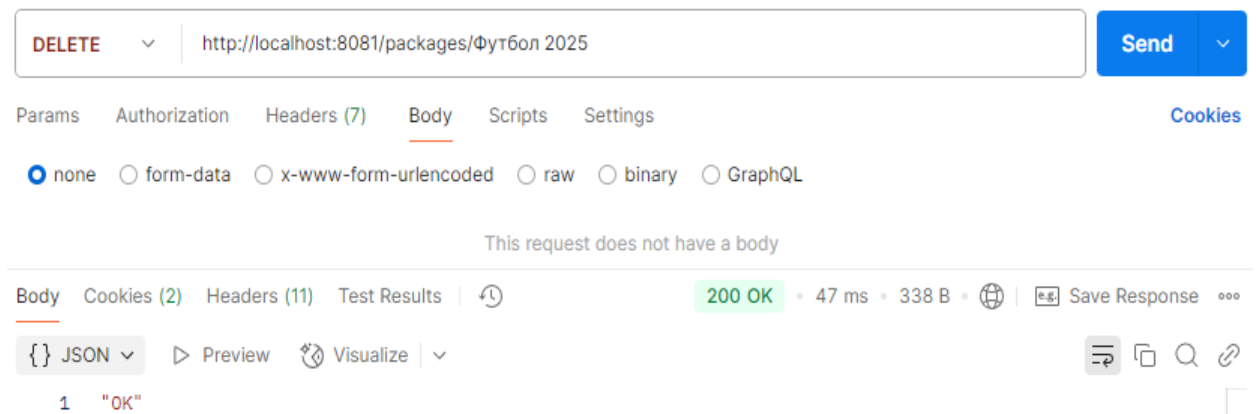


Рисунок 65 - Пример запроса удаления папки, ответ 200

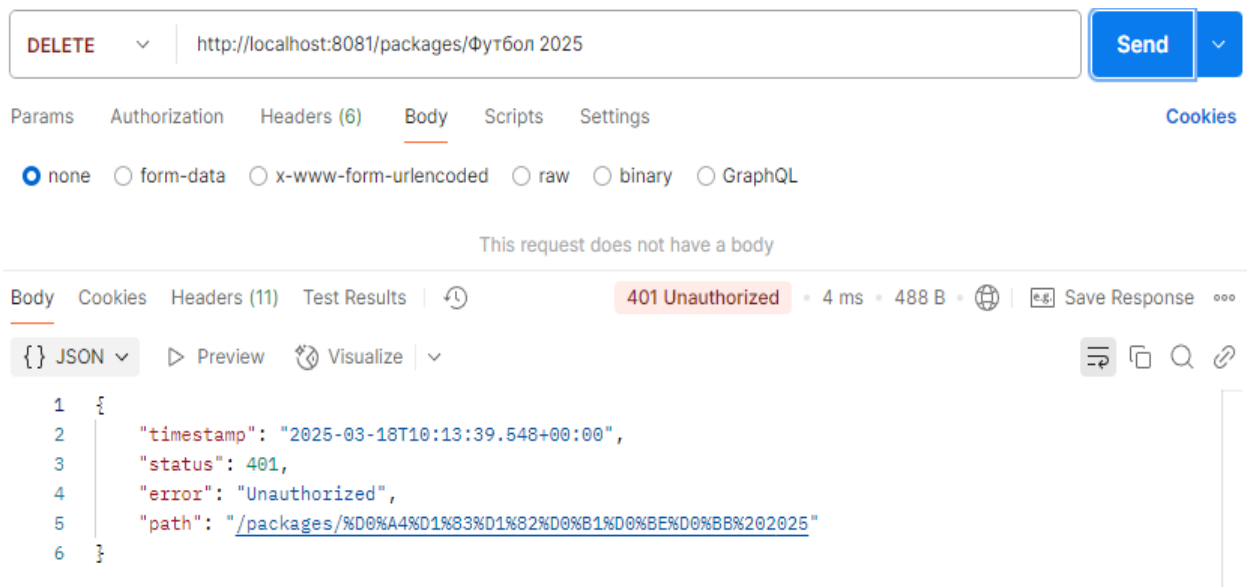


Рисунок 66 - Пример запроса удаления папки, ответ 401

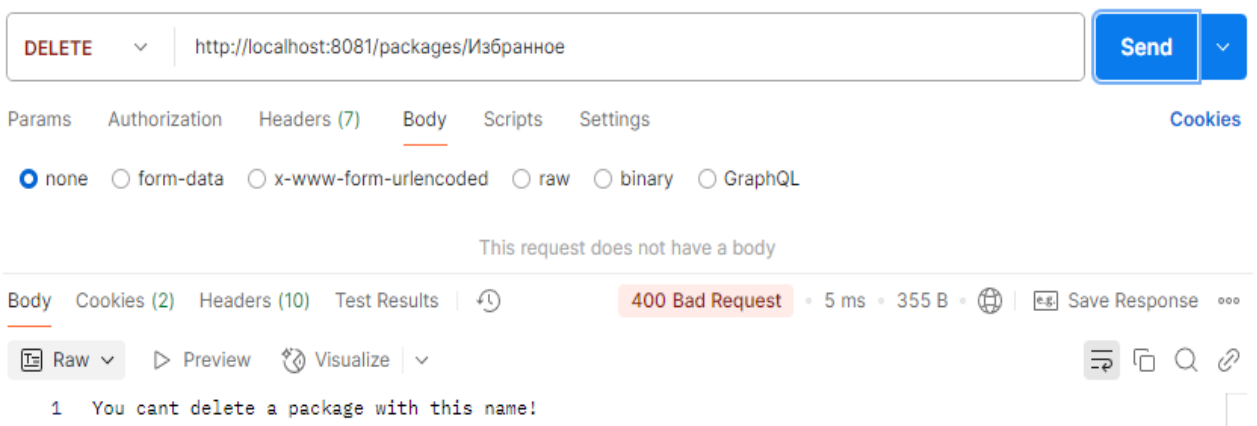


Рисунок 67 - Пример запроса удаления папки, ответ 400

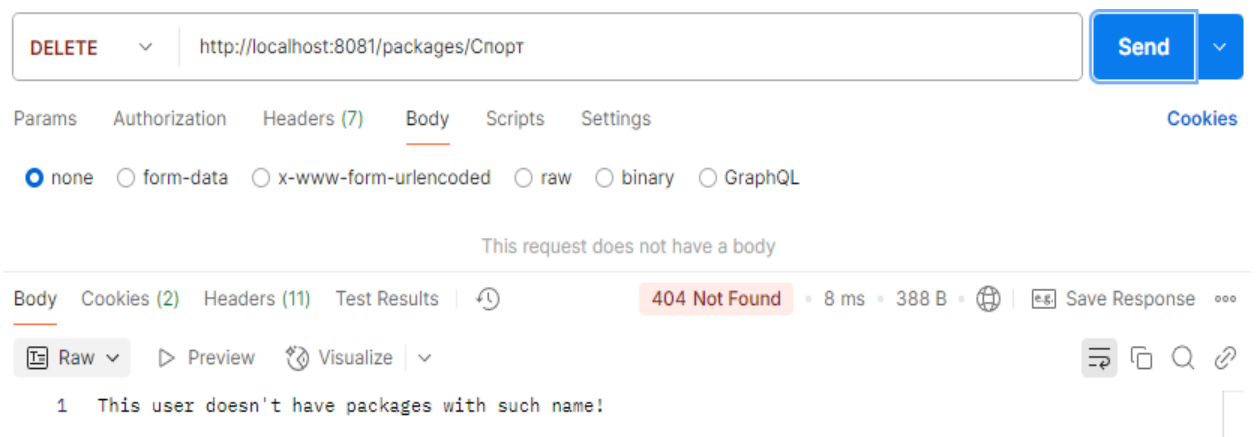


Рисунок 68 - Пример запроса удаления папки, ответ 404

### 3.5 Функционал, связанный с управлением жизненным циклом подписок на дайджесты

Для удобства сначала опишем схемы запросов и ответов, которые могут быть использованы в последующем функционале:

- Схема создания новой подписки на дайджест:
  - 1) “title” – название новой подписки;
  - 2) “description” – описание новой подписки;
  - 3) “sources” – источники, с которых нужно собирать статьи для формирования дайджестов;
  - 4) “tags” – тэги, по которым нужно отбирать статьи для формирования дайджестов;
  - 5) “followers” – логины/почты пользователей, которых создатель подписки желает подписать;
  - 6) “public” – флаг обозначающий, что подписка публичная, то есть видна всем пользователям и доступна для подписывания или нет;
  - 7) “subscribe\_options” – глобальные настройки рассылки для всех подписывающихся и подписанных:
    - a) “send\_to\_mail” – флаг, обозначающий, нужно ли отправлять сформированные дайджесты на почту;
    - б) “mobile\_notifications” – флаг, обозначающий, нужно ли отправлять сформированные дайджесты через уведомления в мобильном приложении.

#### 3.5.1 Создание подписки

Пользователь может выбрать настройки подписки, а также подписать на нее людей. Чтобы создать новую подписку необходимо воспользоваться запросом POST “/subscriptions/create”, передавая в тело запроса параметры в соответствии со схемой создания новой подписки, информация о владельце создателе подписки собирается из access токена, а затем он также добавляется в список подписчиков.

После отправления запроса, возможны следующие варианты ответов, все варианты ответа, кроме 200, ведут к отклонению выполнения запроса:

- 200 – успешно удалось создать новую подписку и подписать на нее пользователей (смотри рисунок 69),
- 401 – попытка создать подписку, не авторизовавшись, что запрещено (смотри рисунок 70),

- 400 – как минимум один источник из списка имеет некорректное имя (смотри рисунок 71),
- 404 – как минимум один пользователь из списка подписчиков не существует в БД (смотри рисунок 72).

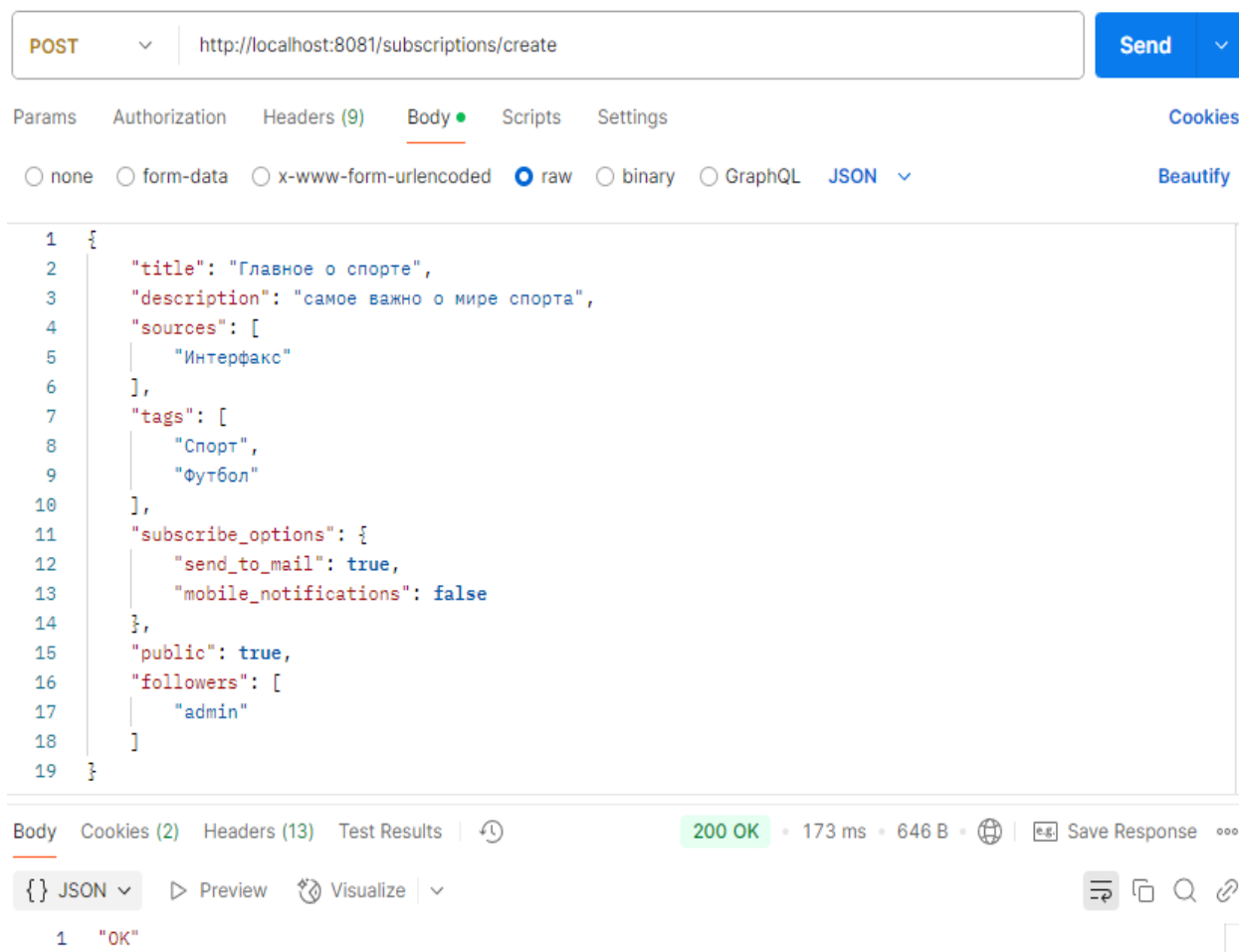


Рисунок 69 - Пример запроса создания подписки на дайджест, ответ 200

POST http://localhost:8081/subscriptions/create Send

Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Beautify

```
1 {
2   "title": "Главное о спорте",
3   "description": "самое важно о мире спорта",
4   "sources": [
5     "Интерфакс"
6   ],
7   "tags": [
8     "Спорт",
9     "Футбол"
10  ],
11  "subscribe_options": {
12    "send_to_mail": true,
13    "mobile_notifications": false
14  },
15  "public": true,
16  "followers": [
17    "admin"
18  ]
19 }
```

Body Cookies Headers (11) Test Results 401 Unauthorized 10 ms 456 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "timestamp": "2025-03-18T14:14:29.079+00:00",
3   "status": 401,
4   "error": "Unauthorized",
```

Рисунок 70 - Пример запроса создания подписки на дайджест, ответ 401

POST http://localhost:8081/subscriptions/create Send

Params Authorization Headers (9) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Beautify

```
1 {
2   "title": "Главное о спорте",
3   "description": "самое важно о мире спорта",
4   "sources": [
5     "Интерфакс",
6     "Некорректный источник"
7   ],
8   "tags": [
9     "Спорт",
10    "Футбол"
11  ],
12  "subscribe_options": {
13    "send_to_mail": true,
14    "mobile_notifications": false
15  },
16  "public": true,
17  "followers": [
18    "admin"
19  ]
20 }
```

Body Cookies (2) Headers (12) Test Results 400 Bad Request 79 ms 719 B Save Response

Raw Preview Visualize

1 Неизвестное русское название: Некорректный источник

Рисунок 71 - Пример запроса создания подписки на дайджест, ответ 400

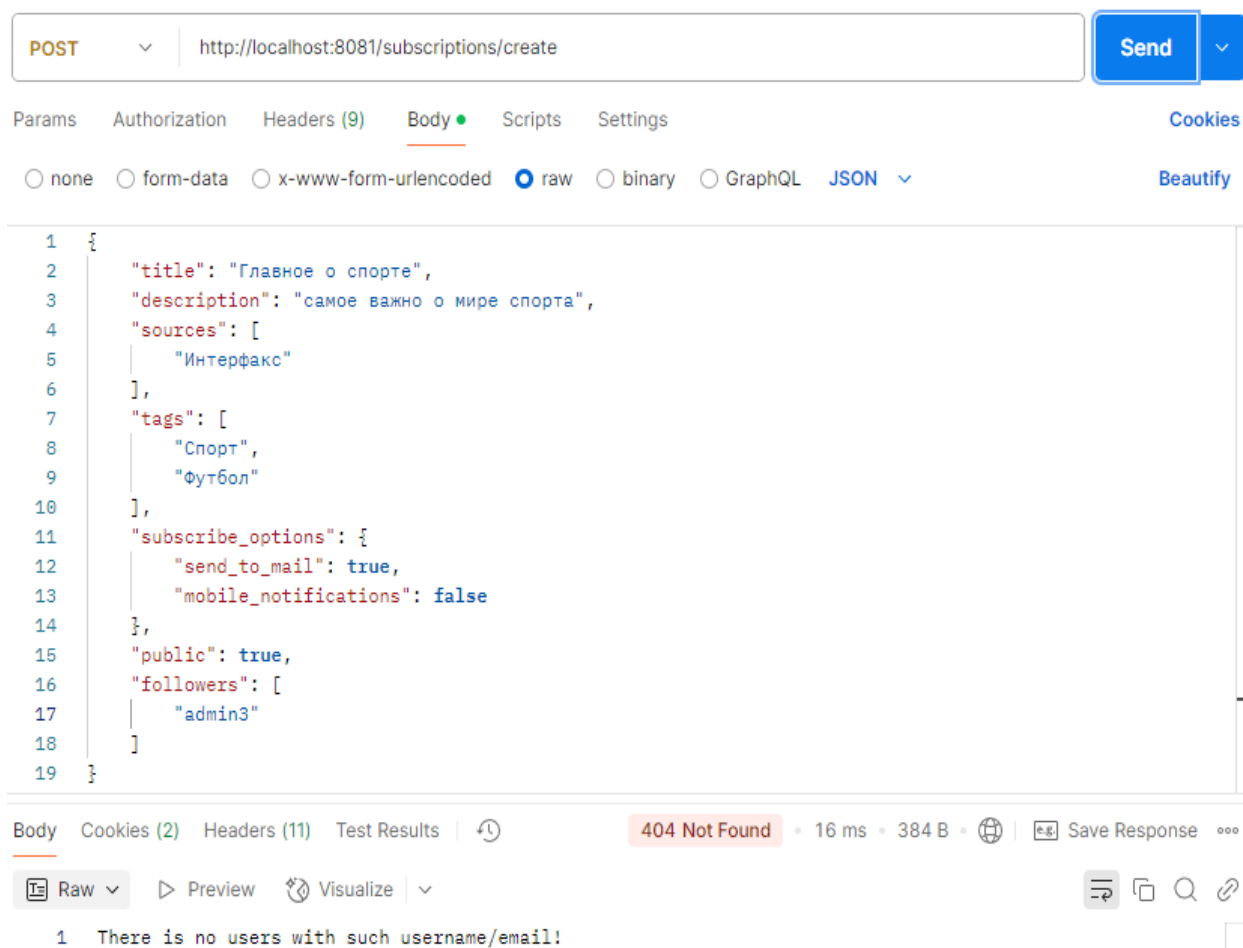


Рисунок 72 - Пример запроса создания подписки на дайджест, ответ 404

### Выводы по главе

В данной главе была детально рассмотрена реализация проекта, с полностью описанными возможными вариантами исхода выполнения запросов. Также для некоторого функционала были добавлены уточнения, чтобы учесть его особенности.

## Заключение

Современная банковская сфера сталкивается с необходимостью оперативного анализа огромных объемов новостных данных для принятия стратегически важных решений. Система «Умный корпус текстов СМИ» разработана для автоматизации процессов сбора, классификации и формирования дайджестов из медиаисточников, что позволяет существенно сократить временные и ресурсные затраты сотрудников Т-банка.

Результатом работы стала реализация серверной части системы, которая обеспечивает хранение и управление новостными статьями, взаимодействие с другими частями Умного корпуса текстов СМИ посредством API.

В рамках исследования проведен анализ предметной области и существующих решений, таких как SmartNews, Feedly, Seldon News и других. Выявлены ключевые потребности Т-банка, включая автоматизацию сбора статей, формирование персонализированных дайджестов и инструменты для работы с контентом. На основе сравнения с аналогами определены уникальные особенности системы, такие как гибкая настройка источников, оценка качества работы моделей классификации, формирования кратких выжимок и новостных дайджестов и реализация инструментов аннотирования, комментирования и группировки по папкам.

Для реализации выбран технологический стек: Java Spring Boot (бизнес-логика), PostgreSQL (хранение метаданных и фильтрация), OpenSearch (поиск).

По окончании работы, система полностью выполняет все требования по функционалу, установленные в начале разработки.

Для дальнейшей развития проекта запланировано дообучение моделей классификации, формирования кратких выжимок и новостных дайджестов на основе собранных оценок пользователей.

## Источники

1. SmartNews [Электронный ресурс] – URL: <https://www.smartnews.com/en> режим доступа: свободный (дата обращения: 19.11.2024).
2. Feedly [Электронный ресурс] – URL: <https://feedly.com/> режим доступа: свободный (дата обращения: 19.11.2024).
3. Seldon News [Электронный ресурс] – URL: <https://news.myseldon.com/en/> режим доступа: свободный (дата обращения: 19.11.2024).
4. uCrawler [Электронный ресурс] – URL: <https://ucrawler.app/ru> режим доступа: свободный (дата обращения: 19.11.2024).
5. Обзор приложения Flipboard: все в одном месте [Электронный ресурс] - URL: <https://geekon.media/obzor-prilozheniya-flipboard/?ysclid=mapg8ail9a279396196> режим доступа: свободный (дата обращения: 18.11.2024)
6. Рамблер новости [Электронный ресурс] - URL: <https://news.rambler.ru/> режим доступа: свободный (дата обращения: 18.11.2024)
7. Яндекс Дзен [Электронный ресурс] - URL: <https://dzen.ru/> режим доступа: свободный (дата обращения: 18.11.2024)
8. PostgreSQL. [Электронный ресурс] – URL: <https://www.postgresql.org/docs/> режим доступа: свободный (дата обращения: 19.11.2024).
9. Java. [Электронный ресурс] – URL: <https://docs.oracle.com/en/java/> режим доступа: свободный (дата обращения: 19.11.2024).
10. Spring. [Электронный ресурс] – URL: <https://docs.spring.io/spring-framework/reference/index.html> режим доступа: свободный (дата обращения: 19.11.2024).
11. Elasticsearch [Электронный ресурс] – URL: <https://www.elastic.co/elasticsearch> режим доступа: свободный (дата обращения: 19.11.2024).
12. OpenSearch [Электронный ресурс] - URL: <https://opensearch.org/> режим доступа: свободный (дата обращения: 19.11.2024)